Java Brew

Auction Website

Software Design Document

Deliverable 3 Addendum

Revisions

Version	Date	Author(s)	Summary of Changes
1.0	1/26/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Initial Version: Creation of the SDD and adding the table of contents and all major sections for organization.
1.1	1/29/26	Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Completed the Introduction and Major Design Decisions sections to the document
1.2	1/31/26	Yazan Haddad, Lama Abdelfattah	Completed and added the Test Driven Development and AI Acknowledgement sections
1.3	2/2/26	Amanullah Asim, Lama Abdelfattah	Completed and added the Sequence Diagram and Activities Diagram sections of the document
1.4	2/3/26	Tristan Augustus, Mahmoud Dahlan, Yazan Haddad	Completed and added the Architecture and Activities Plan sections
1.5	2/6/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Final Version for submission: Updated the revision history and the Group Meeting Logs
2.1	2/18/26	Lama Abdelfattah, Tristan Augustus, Amanullah Asim	Appended the sequence diagrams to better represent the implementations use cases
2.2	2/22/26	Tristan Augustus, Yazan Haddad	Appended activities plan section to better represent what was completed during implementation
2.3	2/25/26	Yazan Haddad, Tristan Augustus, Amanullah Asim	Appended the architecture section to represent the services and interfaces used in the implementation. Appended the diagrams associated with the architecture including class diagrams
2.4	3/1/26	Lama Abdelfattah, Tristan Augustus,	Appended the test case summary to explain the postman test and test cases made
2.5	3/3/26	Mahmoud Dahlan, Tristan Augustus	Appended key design choices to explain the 4 design patterns used and 2 unused and appended sections 11 and 12
2.6	3/6/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Final Version for submission updated/appended revision history and meeting log
3.1	3/14/26	Yazan Haddad, Lama	Appended the docker deployment to

		Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	summarize what was added and how the docker was deployed
3.2	3/23/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus	Appended the test plan summary to include the JMeter and Postman tests and results
3.3	3/26/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Amanullah Asim	Added and appended the distinguishable features section to include the Role-based access summary and AI Chatbot summary
3.4	3/31/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Appended the AI advantages and disadvantages sections as well as the AI acknowledgments section. Also appended the amendments section to include what was done for deliverable 3 and appended the team member parts section to give an overview of what each team member did.

Table of Contents

Revisions....2
Table of Contents.... 3
1 Introduction....5
1.1 Purpose.... 5
1.2 Overview....5
1.3 References.... 5
2 Major Design Decisions.... 5
Key Design Choices.... 5
(Appended) Key Design Choices....6
3 Sequence Diagrams.... 7
UC1: Sign Up and Sign In.... 7
UC5: Payment.... 9
UC2.1: Search Auction Items....11
UC3: Place a Bid.... 12
3 (Appended) Sequence Diagrams.... 12
UC1: Sign-up and Login.... 12

UC2: Browse and Search Catalogue....14
UC3: Placing and Validating Bids....14
UC4: Auction End Behaviour....15
UC5: Payment....17
UC6: Receipt and Shipping....18
UC7: Creating An Auction Listing....19
4 Activity Diagrams....20
4 (Appended)Activity Diagrams....23
UC1: Sign-up and Login....24
UC2: Browse and Search Catalogue....25
UC3: Placing and Validating Bids....26
UC4: Auction End Behaviour....27
UC5: Payment....28
UC6: Receipt and Shipping....29
UC7: Creating An Auction Listing....30
5 Architecture....31
5.1 Decomposition Diagram....31
5.2 Services Table....31
5.3 Interfaces Table....32
5 (Appended) Architecture....33
5.1 (Appended) Decomposition Diagram....33
5.2 (Appended) Services Table....34
5.3 (Appended) Interfaces Table....34
6 Activities Plan....37
6.1 Product Backlog for Auction System....37
6.1 (Appended) Product Backlog for Auction System....38
6.2 Deliverable 1 Sprint Backlog....39
6.3 Deliverable 2 Sprint Backlog....39
6.3 (Appended) Deliverable 2 Sprint Backlog....40
6.4 (Appended) Deliverable 3 Sprint Backlog....40
6.5 Gantt Diagram....41
7 (Appended) Group Meeting Logs....41
7 (Appended) Group Meeting Logs....41
8 (Appended) Test-driven development....42
9 (Appended) Test plan Summary....42
9.1 (Appended) Postman and JUnit Test Cases Added (See GitHub)....42
9.2 (Appended) JMeter Performance Test Results....43
10 (Appended) Distinguishable Feature....44
10.1 Role-Based Access Control....44
10.2 AI Chatbot - Admin Use....44
11 (Appended) Frontend Implementation....44
12 (Appended) Docker Deployment....45
13 (Appended) AI Advantages and Disadvantages....45
Advantages:....46

Disadvantages:.....46
14 (Appended) AI Acknowledgement.....46
15 (Appended) Deviations from Original Design.....47
16 (Appended) Amendments to the Original Document.....47
17 (Appended) Team Member Parts.....48

1 Introduction

1.1 Purpose

The purpose of this project is to design and later implement a web-based auction e-commerce system that supports forward auctions. The system allows users to register, browse auction items, place bids in real time, and complete payments for items they win. This Software Design Document (SDD) defines the high-level design, architecture, and development plan for the system.

1.2 Overview

The auction platform enables sellers to list items for sale using forward auctions, where bidders compete by placing increasingly higher bids until a fixed auction time expires. Users can sign up and sign in, search for auction items, view current bid prices and remaining time, and place bids on selected items. When an auction ends, the highest bidder is declared the winner and proceeds to payment. After payment is completed, the system provides a receipt and shipping details.

The system is designed using modular services to ensure scalability, maintainability, and ease of future deployment as microservices. This deliverable focuses on design and planning only; implementation will be completed in later deliverables.

1.3 References

EECS 4413 Project Description Document – Auction E-Commerce System
UML 2.x Specification

2 Major Design Decisions

Key Design Choices

The system is designed as a multi-tier web application consisting of a front-end user interface, a gateway service, and several back-end services responsible for authentication, catalogue management, auctions, and payments. Separating responsibilities into dedicated services simplifies development and allows independent scaling of system components.

Modularity: High Cohesion and Low Coupling

Each service in the system has a single well-defined responsibility:

- The **IAM Service** handles user registration and authentication.
- The **Catalogue Service** manages auction item listings and search.
- The **Auction Service** handles bidding logic and auction timing.
- The **Payment Service** manages payment processing and receipt generation.

This separation ensures high cohesion, since each module focuses on one functional area, and low coupling, since services interact only through clearly defined interfaces rather than direct internal dependencies.

Chosen Architectural Pattern

A **Layered Service-Oriented Architecture (SOA)** combined with a **Gateway (Facade) Pattern** is chosen.

- The UI layer handles user interaction.
- The Gateway Service acts as a single entry point that routes requests to back-end services.
- The Back-End Services layer contains IAM, Catalogue, Auction, and Payment services.
- The Data layer stores persistent information in databases.

This architecture improves maintainability, supports future microservice deployment, and allows services to evolve independently.

Rejected Architectural Pattern

A **Monolithic Architecture** was considered but rejected. In a monolithic design, all system components would be tightly integrated into a single application. This would make future scaling difficult, increase maintenance complexity, and reduce flexibility in deploying services independently. Therefore, a modular service-based approach was selected instead.

(Appended) Key Design Choices

Design Patterns Used

The implementation of the JavaBrew auction backend applies several design patterns to improve modularity, maintainability, and clarity.

1. Layered Architecture Pattern
The backend was implemented using a layered structure consisting of Controllers, Services, and Repositories. Controllers handle HTTP requests and responses, Services contain the core business logic, and Repositories manage persistence. This separation improves maintainability and ensures that each layer has a clear responsibility.

2. Repository Pattern
The Repository pattern was used through Spring Data JPA repositories such as UserRepository, ItemRepository, BidRepository, and PaymentRepository. This pattern abstracts database access from the service layer and allows persistence logic to remain separated from business logic.

3. DTO (Data Transfer Object) Pattern
DTOs were used to separate the internal persistence model from the API request and response structures. Request objects such as SignupRequest, LoginRequest, BidRequest, and PaymentRequest, as well as response objects such as LoginResponse, PaymentResponse, and ReceiptResponse, help isolate external API contracts from internal entities.

4. Dependency Injection Pattern
Dependency Injection was used extensively through Spring Boot to inject services and repositories into controllers and service classes. This reduces coupling between classes and improves testability, maintainability, and flexibility of the application.

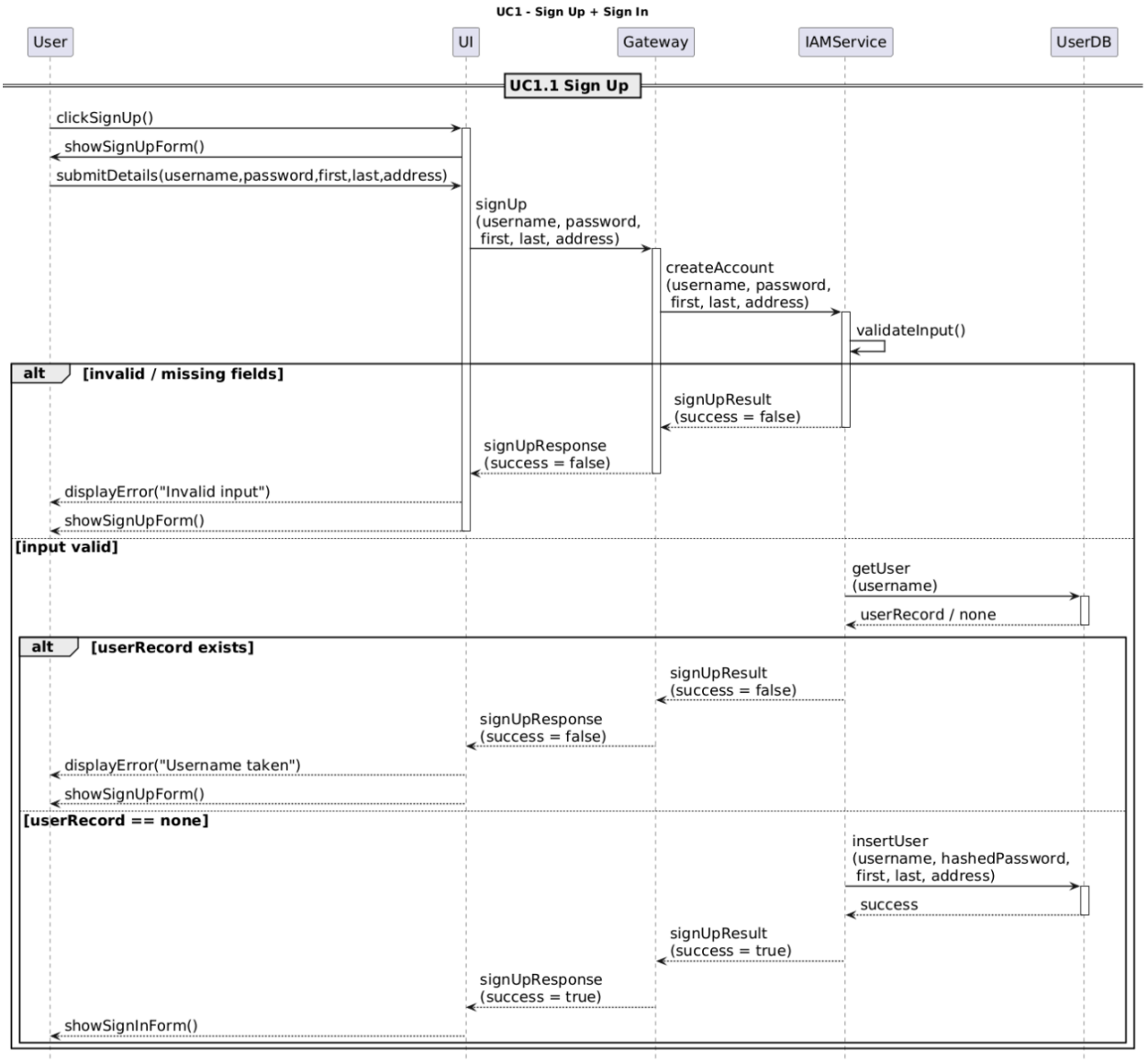
Design Patterns Considered but Not Used

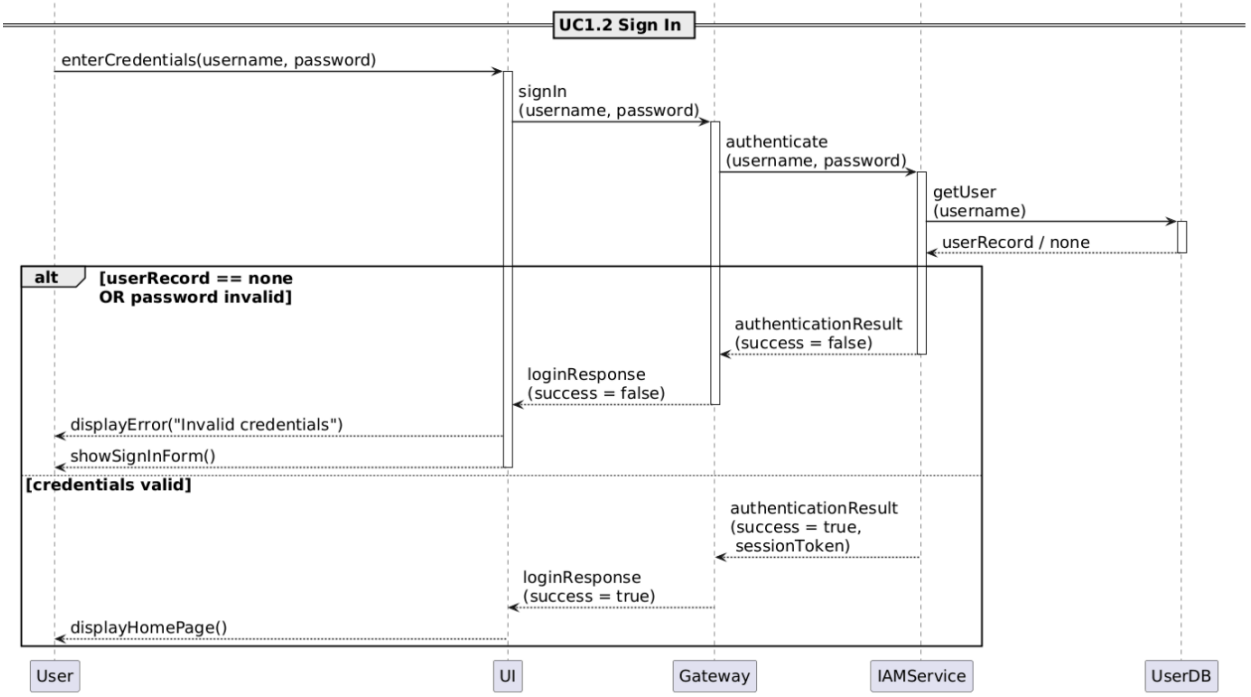
1. Observer Pattern
The Observer pattern was considered for supporting real-time auction notifications, such as notifying users when an auction ends or when a new highest bid is placed. However, this was not implemented in Deliverable 2 because asynchronous notifications were not required for this milestone.

2. Singleton Pattern
The Singleton pattern was considered for services managing application-wide state. However, Spring Boot already manages service and repository beans with singleton scope by default, so implementing a separate Singleton pattern was unnecessary.

3 Sequence Diagrams

UC1: Sign Up and Sign In


UC1 - Sign Up + Sign In
User
UI
Gateway
IAMService
UserDB
UC1.1 Sign Up
clickSignUp()
showSignUpForm()
submitDetails(username,password,first,last,address)
signUp
(username, password,
first, last, address)
createAccount
(username, password,
first, last, address)
validateInput()
alt
[invalid / missing fields]
signUpResult
(success = false)
signUpResponse
(success = false)
displayError("Invalid input")
showSignUpForm()
[input valid]
getUser
(username)
userRecord / none
alt
[userRecord exists]
signUpResult
(success = false)
signUpResponse
(success = false)
displayError("Username taken")
showSignUpForm()
[userRecord == none]
insertUser
(username, hashedPassword,
first, last, address)
success
signUpResult
(success = true)
signUpResponse
(success = true)
showSignInForm()

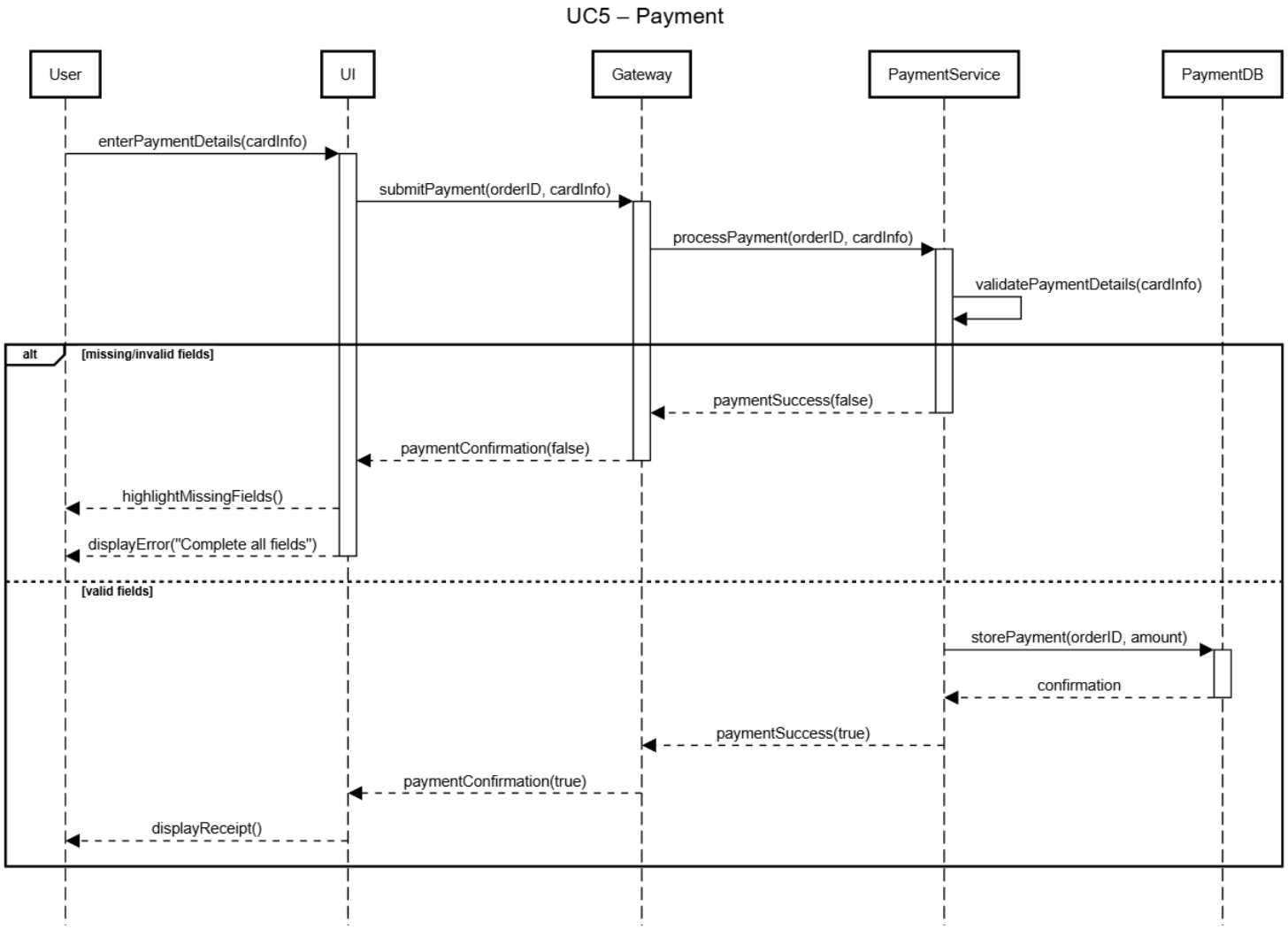


This sequence diagram models the complete user authentication workflow, including both sign-up (UC1.1) and sign-in (UC1.2). During sign-up, the user submits registration details through the UI, which sends requests to the Gateway and then to the IAMService. The IAMService validates input, checks for existing users in the UserDB, and either rejects the request or creates a new user record. Upon successful registration, the user is redirected to the sign-in form.

For sign-in, the user submits credentials that are routed through the Gateway to the IAMService for authentication. The IAMService retrieves user data from the UserDB and verifies the credentials. If authentication fails, an error message is displayed and the user is prompted to retry. If authentication succeeds, a session token is generated and the user is granted access to the home page.

For additional clarity, a high-resolution version of the same diagram has also been uploaded to Google Drive to allow reviewers to view the diagram at full scale if needed using the following link: Here!

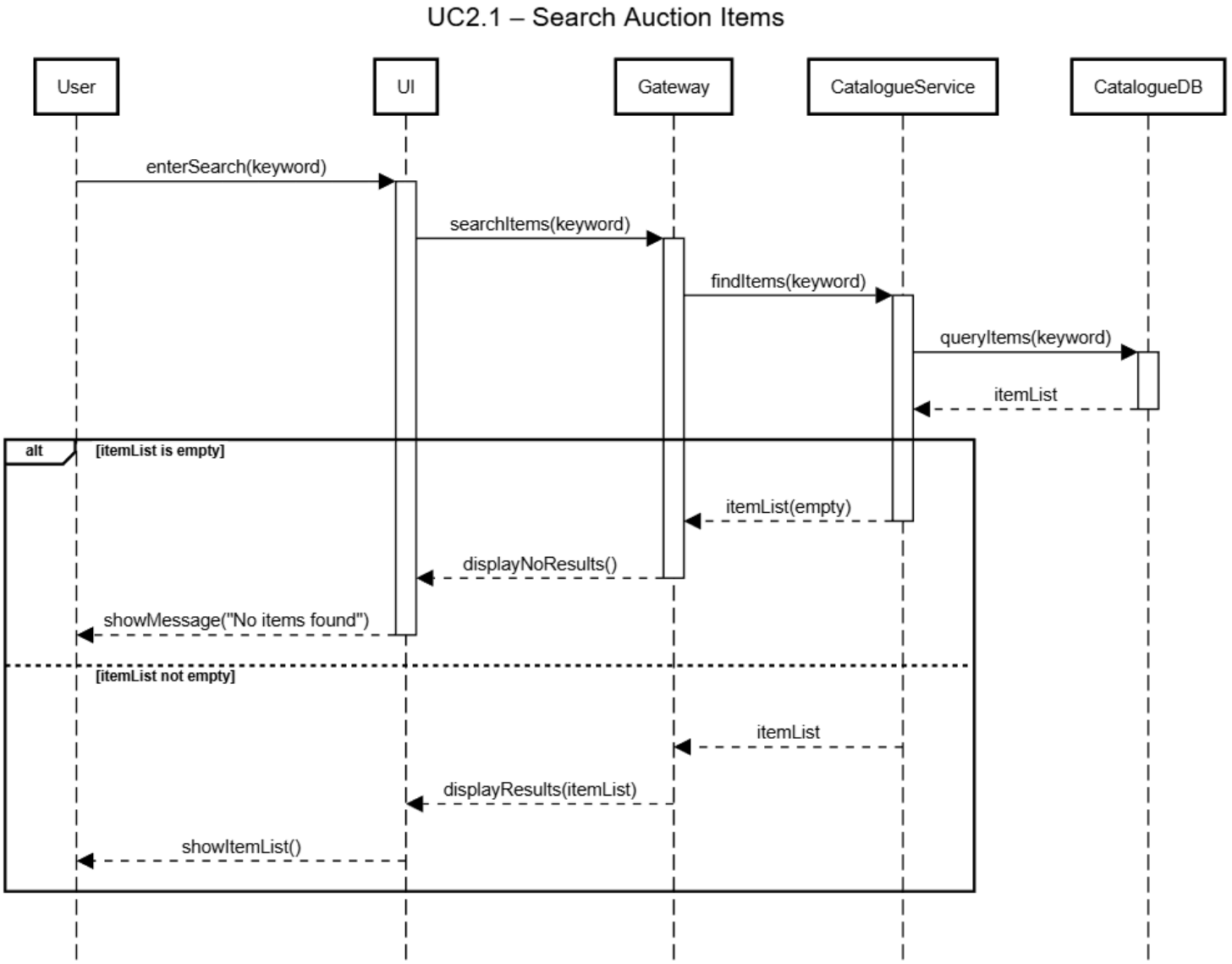
UC5: Payment



This diagram depicts the payment process after an auction has ended. The user enters payment details through the UI, which forwards the request to the Gateway and then to the PaymentService. The PaymentService first validates the payment details.

If required fields are missing or invalid, the payment is rejected and the UI highlights the missing information and displays an error message. If the payment details are valid, the transaction is stored in the PaymentDB and a confirmation is returned. The UI then displays the payment receipt to the user, completing the payment workflow.

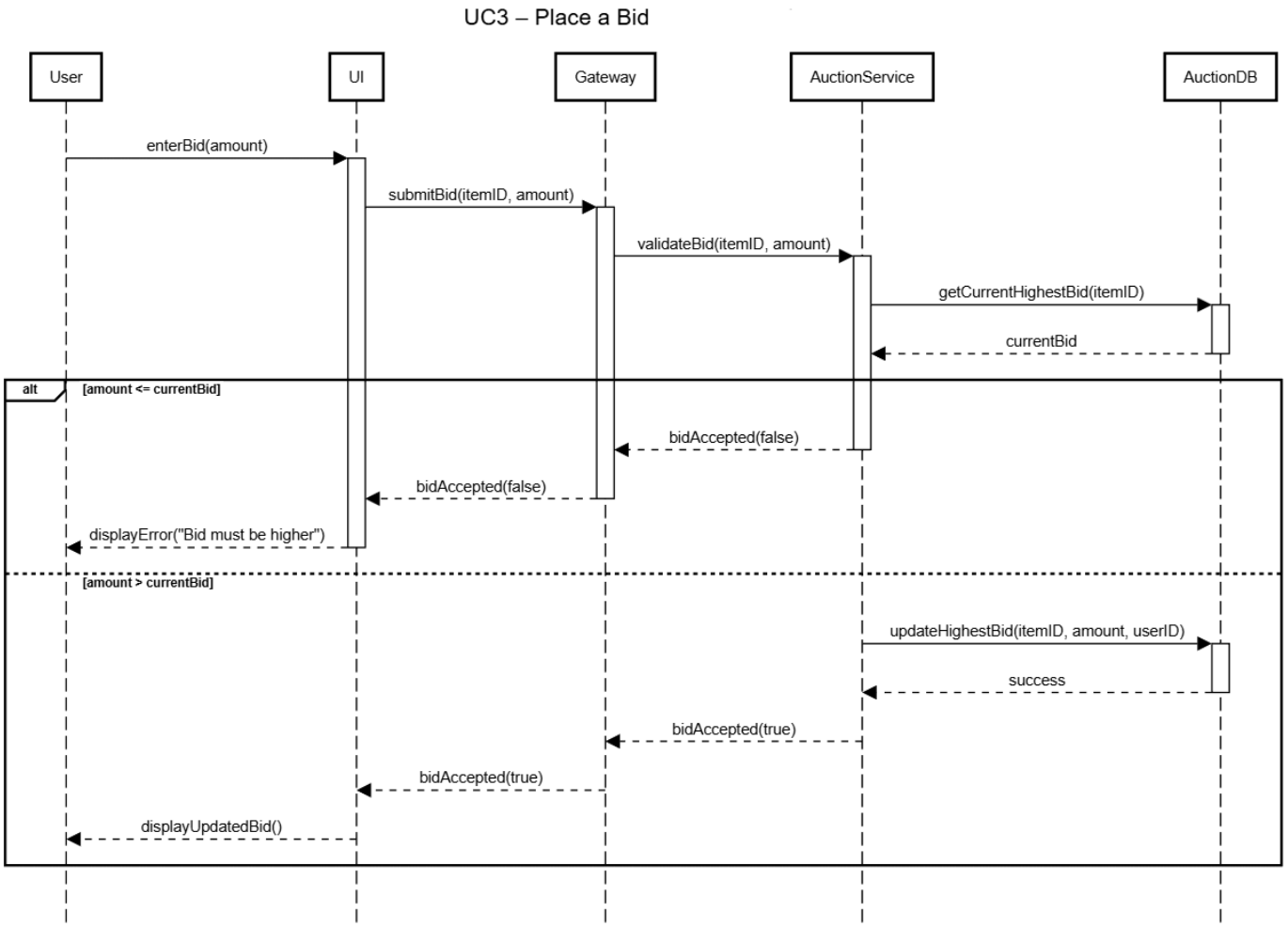
UC2.1: Search Auction Items



This diagram illustrates the auction item search process. The user enters a search keyword through the UI, which sends the request to the Gateway. The Gateway delegates the request to the CatalogueService, which queries the CatalogueDB for matching auction items.

Two outcomes are modeled: if no items match the search criteria, an empty result set is returned and the UI displays a “No items found” message. If matching items exist, the item list is returned to the UI and displayed to the user. This diagram demonstrates proper separation of concerns, where the UI handles presentation, services handle business logic, and the database manages persistence.

UC3: Place a Bid

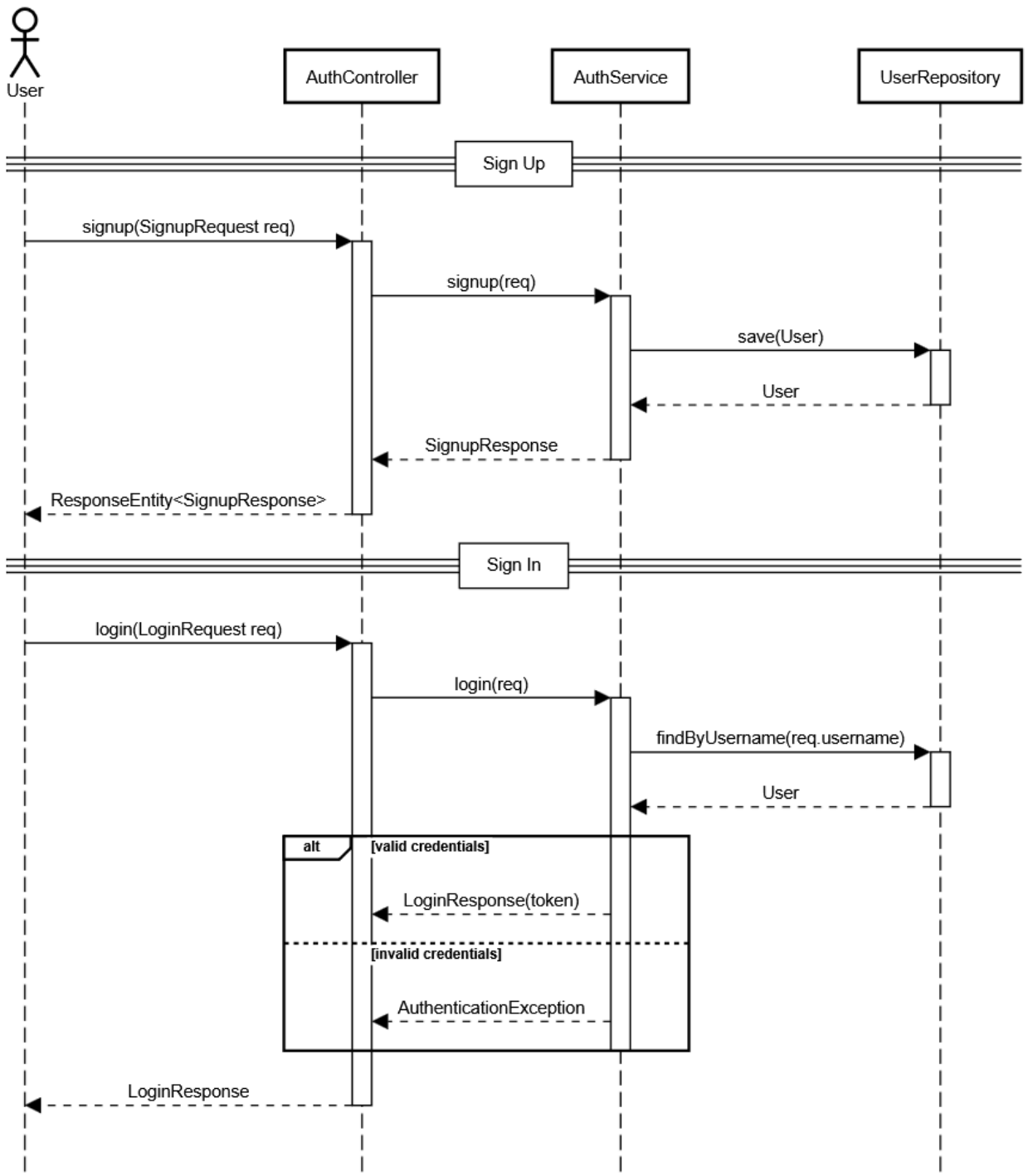


This sequence diagram represents the bidding process for a forward auction. The user submits a bid amount via the UI, which sends the request to the Gateway and subsequently to the AuctionService. The AuctionService retrieves the current highest bid from the AuctionDB and validates the new bid amount.

If the submitted bid is less than or equal to the current highest bid, the bid is rejected and an error message is displayed to the user. If the bid is valid, the AuctionService updates the highest bid in the database and returns a success response. The updated bid information is then displayed to the user through the UI.

3 (Appended) Sequence Diagrams

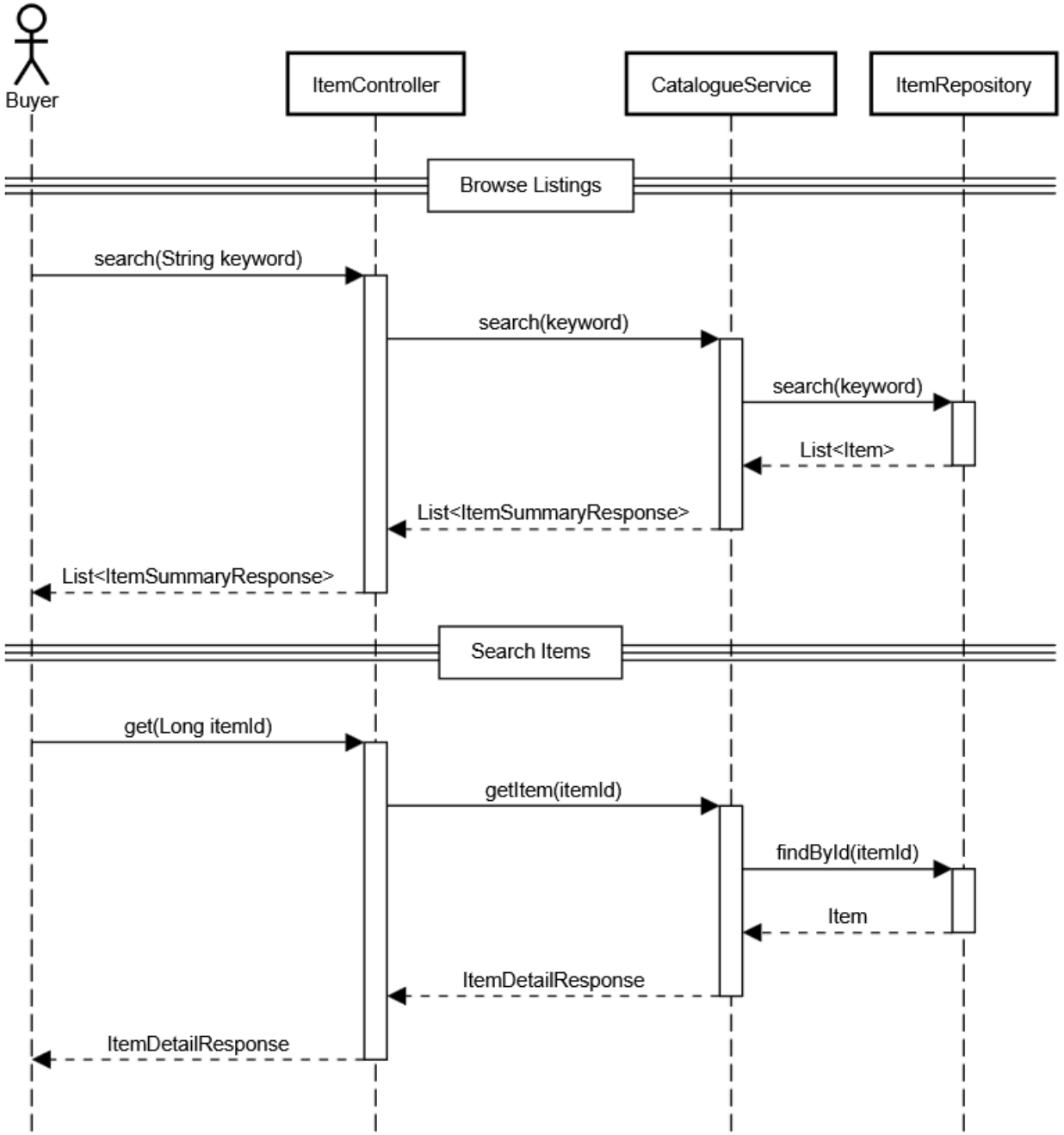
UC1: Sign-up and Login



This sequence diagram models the user authentication workflow, including both account creation and login. During sign-up, the user sends a registration request to the AuthController, which forwards the request to the AuthService. The service validates the input and stores the new user by calling UserRepository.save(). If the registration is successful, a response confirming the account creation is returned to the user.

For login, the user submits their credentials through the AuthController. The controller calls AuthService.login(), which retrieves the user from the database using UserRepository.findByUsername(). The service verifies the credentials and determines whether authentication succeeds or fails. If the credentials are valid, a LoginResponse is returned and the user is authenticated; otherwise, an error message is returned.

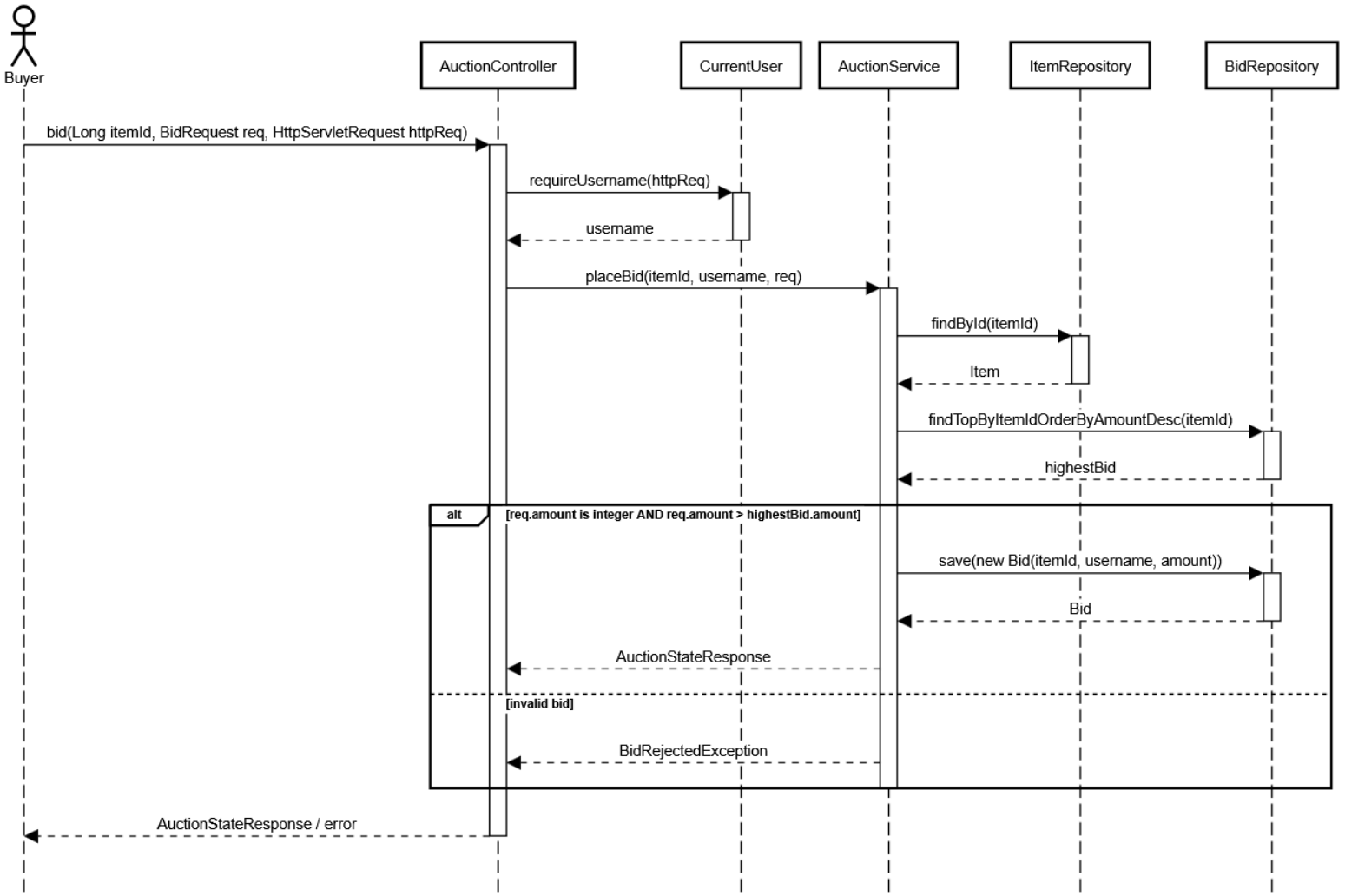
UC2: Browse and Search Catalogue



This sequence diagram models how buyers browse or search for items in the auction catalogue. When a buyer searches for items, a request is sent to the ItemController, which forwards the query to the CatalogueService. The service calls ItemRepository.search() to retrieve matching items from the database and converts them into response objects for the buyer.

If the buyer selects a specific item, the ItemController calls CatalogueService.getItem() to retrieve detailed item information. The service fetches the item using ItemRepository.findById(). The resulting item details are returned to the controller and displayed to the buyer.

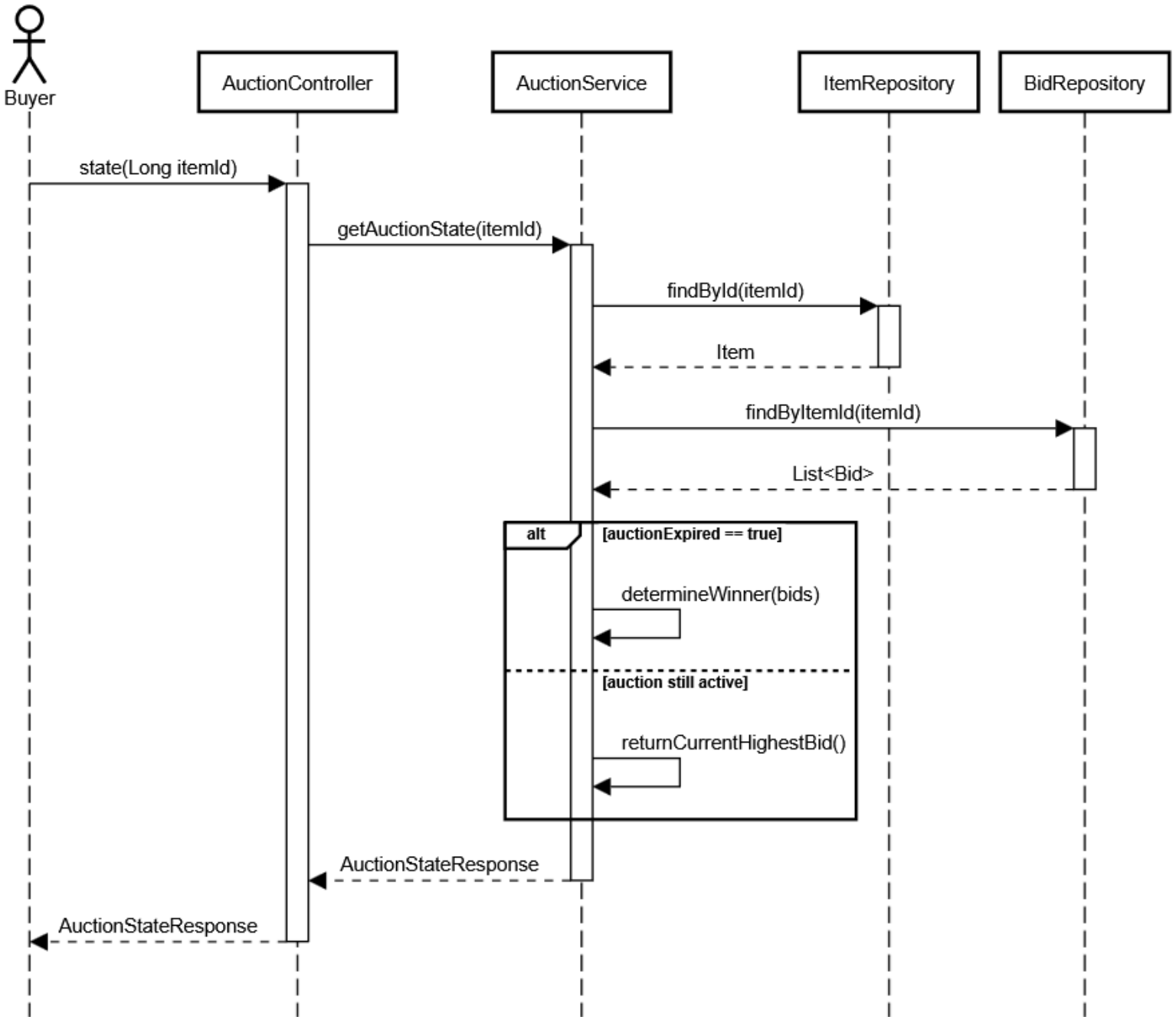
UC3: Placing and Validating Bids



This sequence diagram models the process of placing a valid bid during an auction. The buyer submits a bid request to the AuctionController, which first identifies the logged-in user using CurrentUser.requireUsername(). The controller then calls AuctionService.placeBid() to process the bid.

The service retrieves the item and the current highest bid using ItemRepository.findById() and BidRepository.findTopByItemIdOrderByAmountDesc(). The bid is validated to ensure the amount is an integer and greater than the current highest bid. If the bid is valid, it is saved using BidRepository.save() and the updated auction state is returned. If the bid is invalid, the request is rejected and an error response is returned.

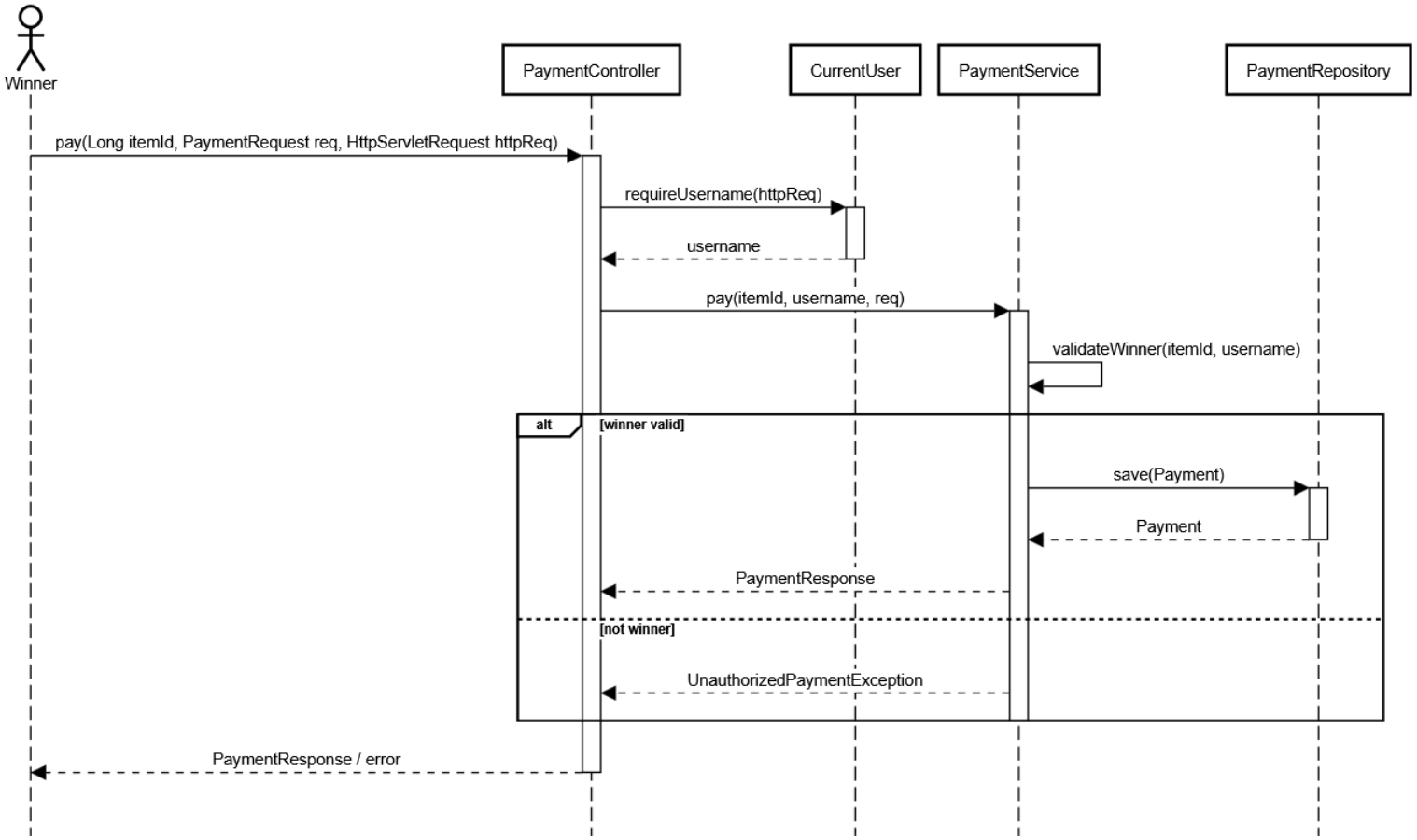
UC4: Auction End Behaviour



This sequence diagram models how the system determines the state of an auction and identifies the winner when the auction ends. A buyer requests the auction status through the AuctionController, which calls AuctionService.getAuctionState().

The service retrieves the item and its associated bids using ItemRepository.findById() and BidRepository.findByItemId(). It then checks whether the auction has expired. If the auction has ended, the highest bid is selected as the winning bid; otherwise, the current auction state is returned. The resulting auction information is sent back to the buyer.

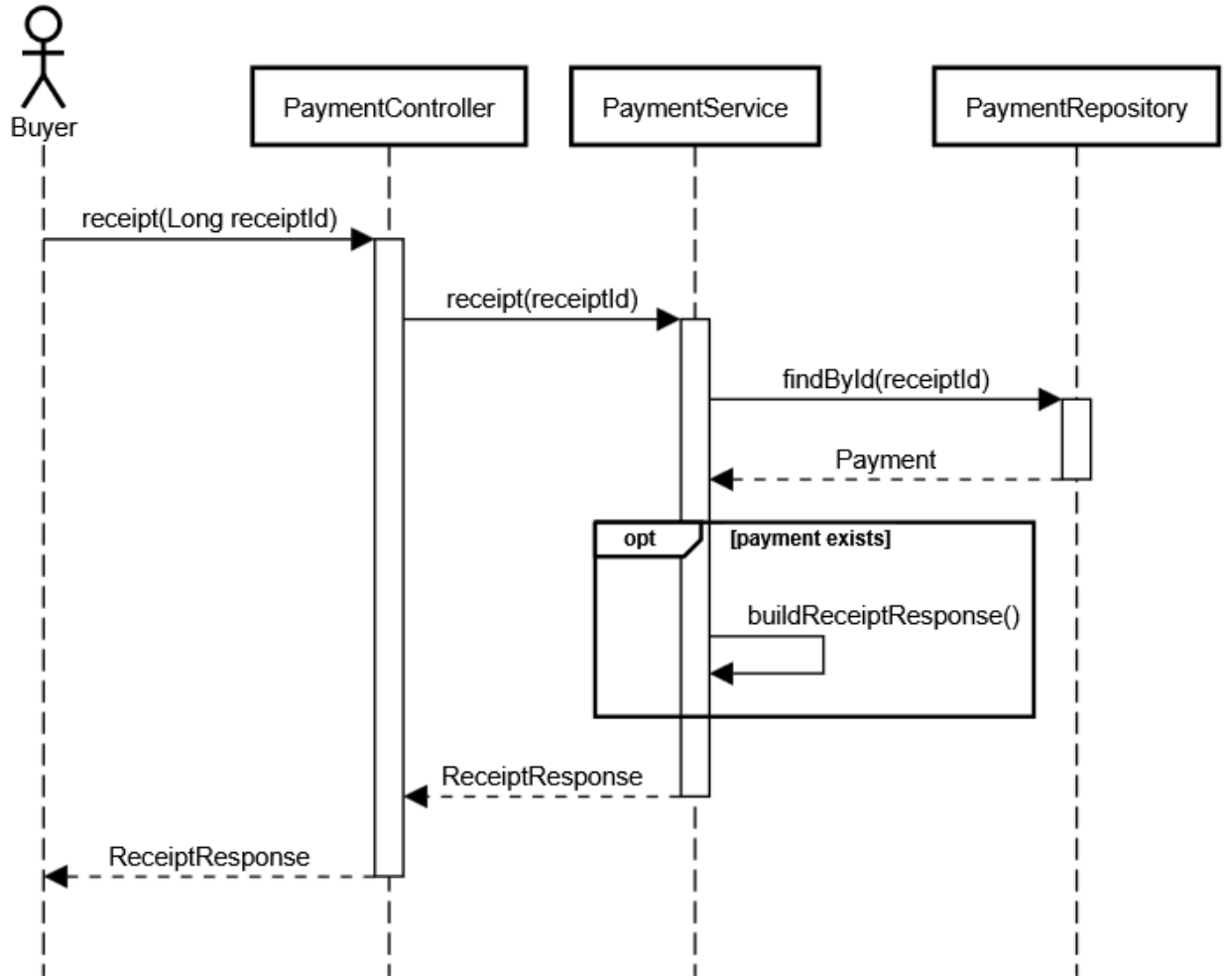
UC5: Payment



This sequence diagram models how the system processes payment for an auction item. The winning bidder submits a payment request to the PaymentController, which identifies the authenticated user using CurrentUser.requireUsername(). The controller then calls PaymentService.pay() to handle the payment.

The service verifies that the user attempting to pay is the winner of the auction. If the validation succeeds, a payment record is stored using PaymentRepository.save(). A PaymentResponse confirming the transaction is then returned to the user. If the user is not the winning bidder, the payment request is rejected.

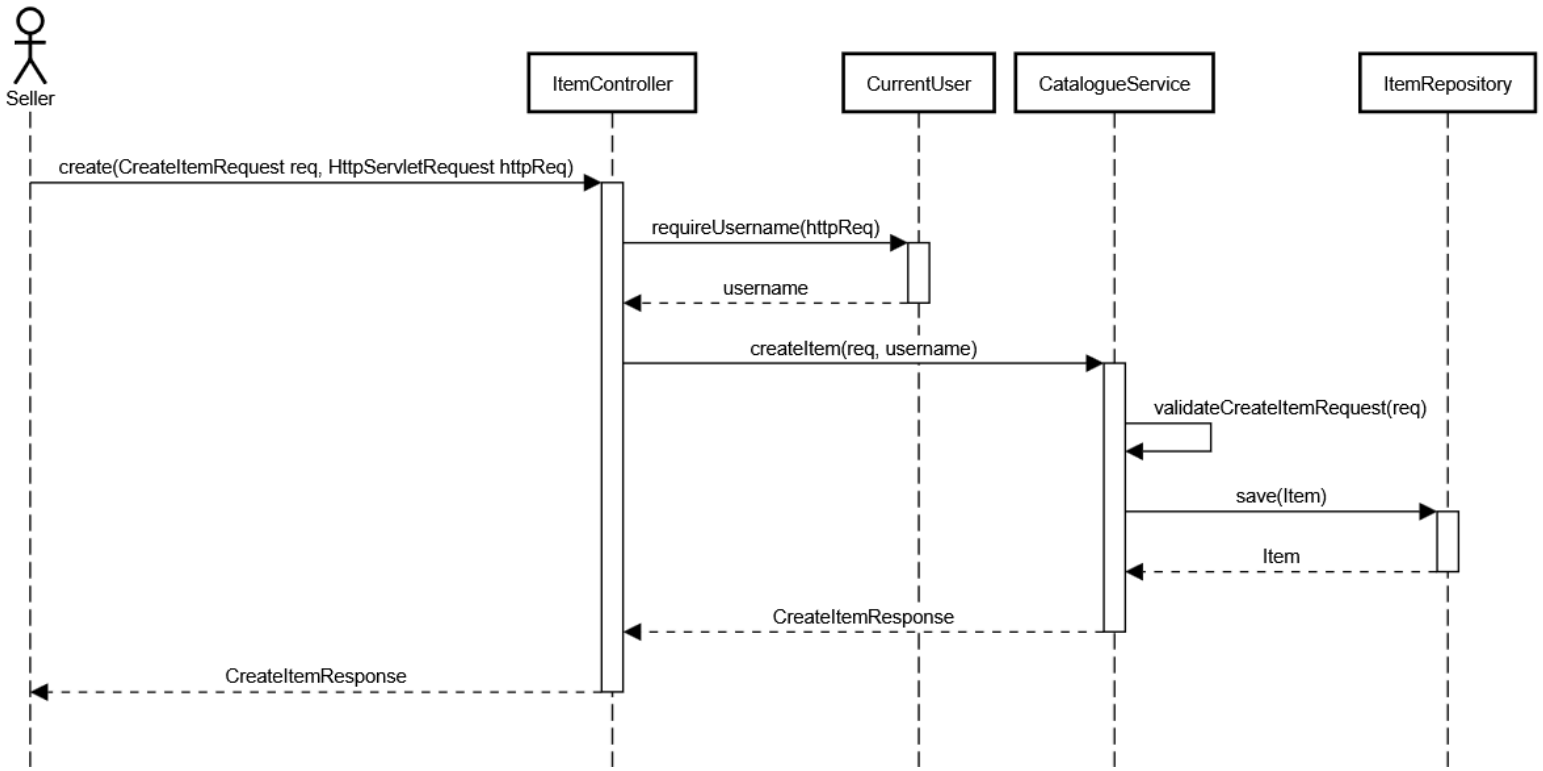
UC6: Receipt and Shipping



This sequence diagram models how the system generates a receipt after a payment is completed. The buyer requests a receipt through the PaymentController using a receipt identifier. The controller forwards the request to the PaymentService.

The service retrieves the payment information using PaymentRepository.findById(). It then constructs a ReceiptResponse containing the transaction details. The receipt is returned to the controller and displayed to the buyer as confirmation of the purchase.

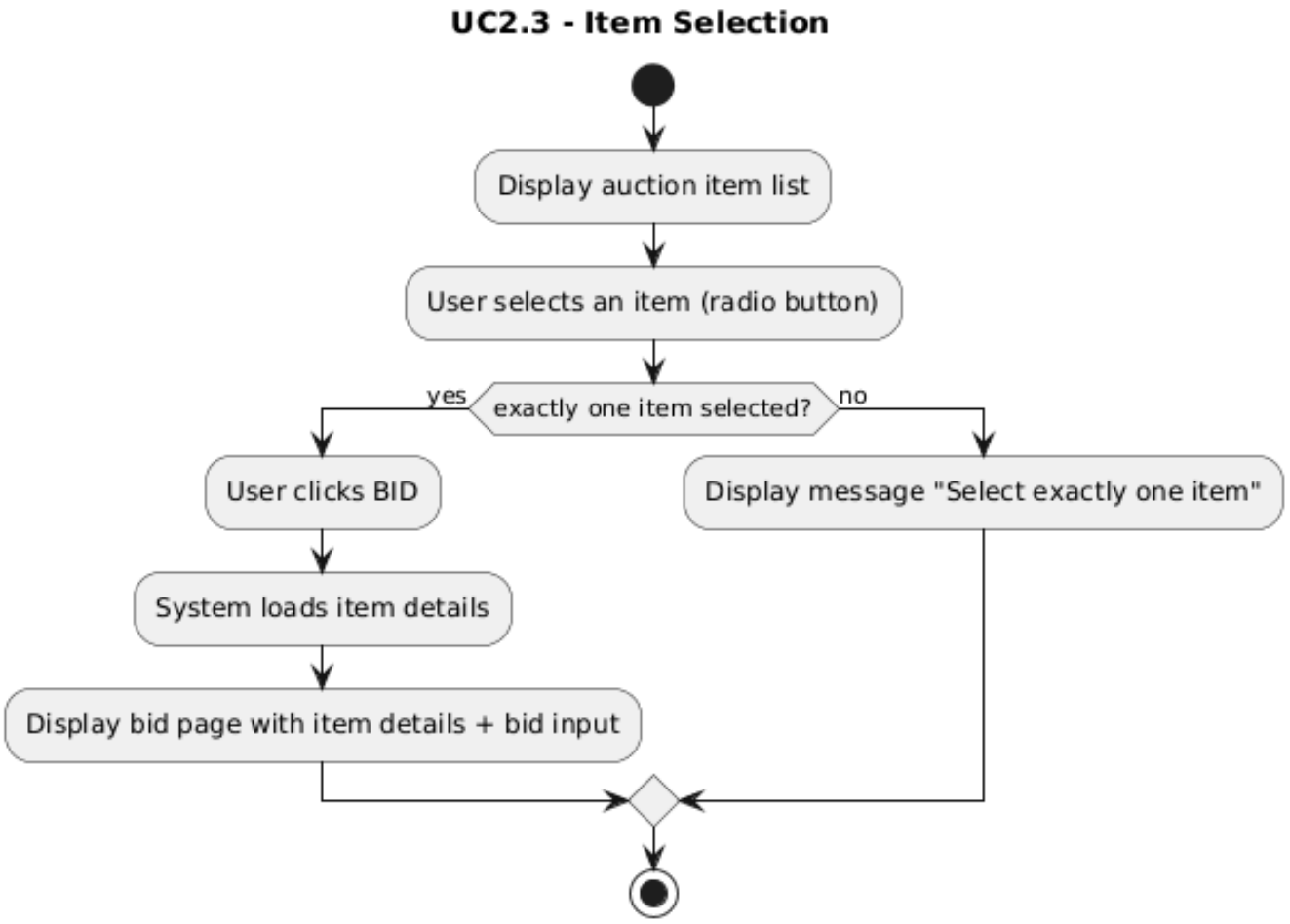
UC7: Creating An Auction Listing



This sequence diagram models the process of creating a new auction listing. The seller submits an item creation request to the ItemController. The controller identifies the authenticated seller using CurrentUser.requireUsername() and forwards the request to CatalogueService.createItem().

The service validates the item information and creates an Item object. The item is stored in the database using ItemRepository.save(). Once the item is successfully saved, a CreateItemResponse is returned to the seller confirming that the listing has been created.

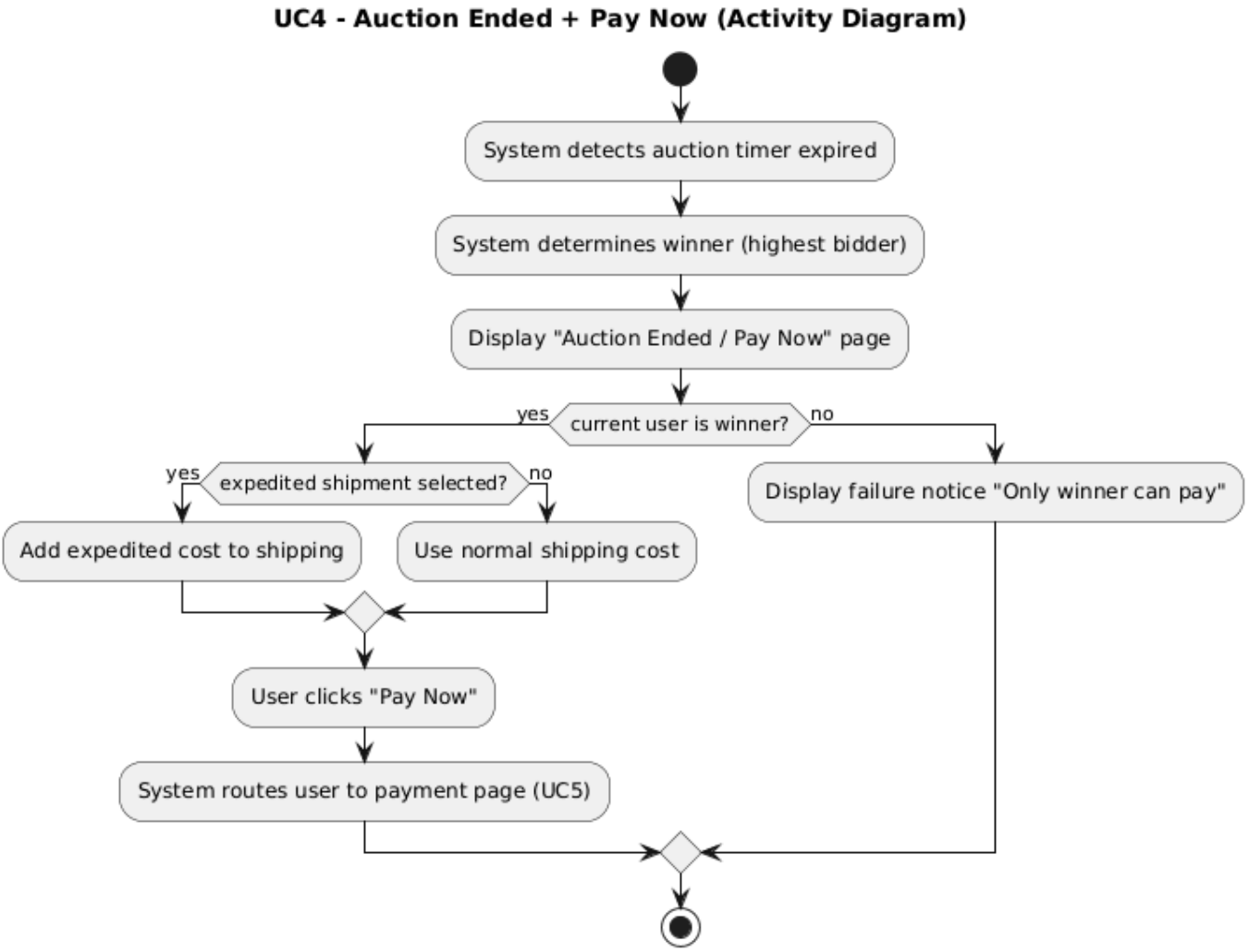
4 Activity Diagrams



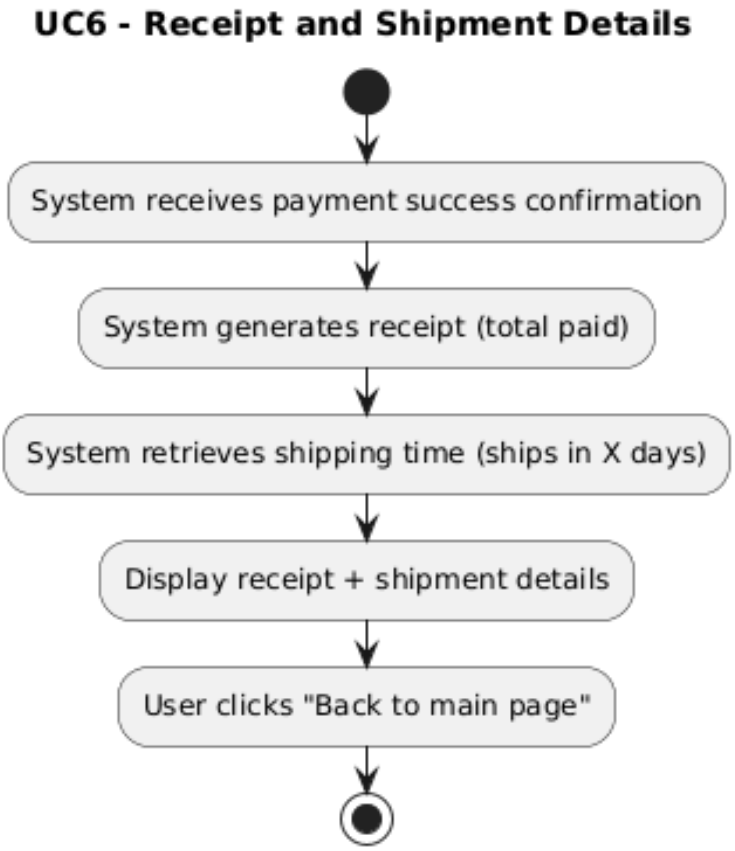
Legend for all Activity diagrams :

- Actions are rounded rectangles; diamonds represent decisions with guards; arrows represent control flow; start/end nodes indicate workflow boundaries.

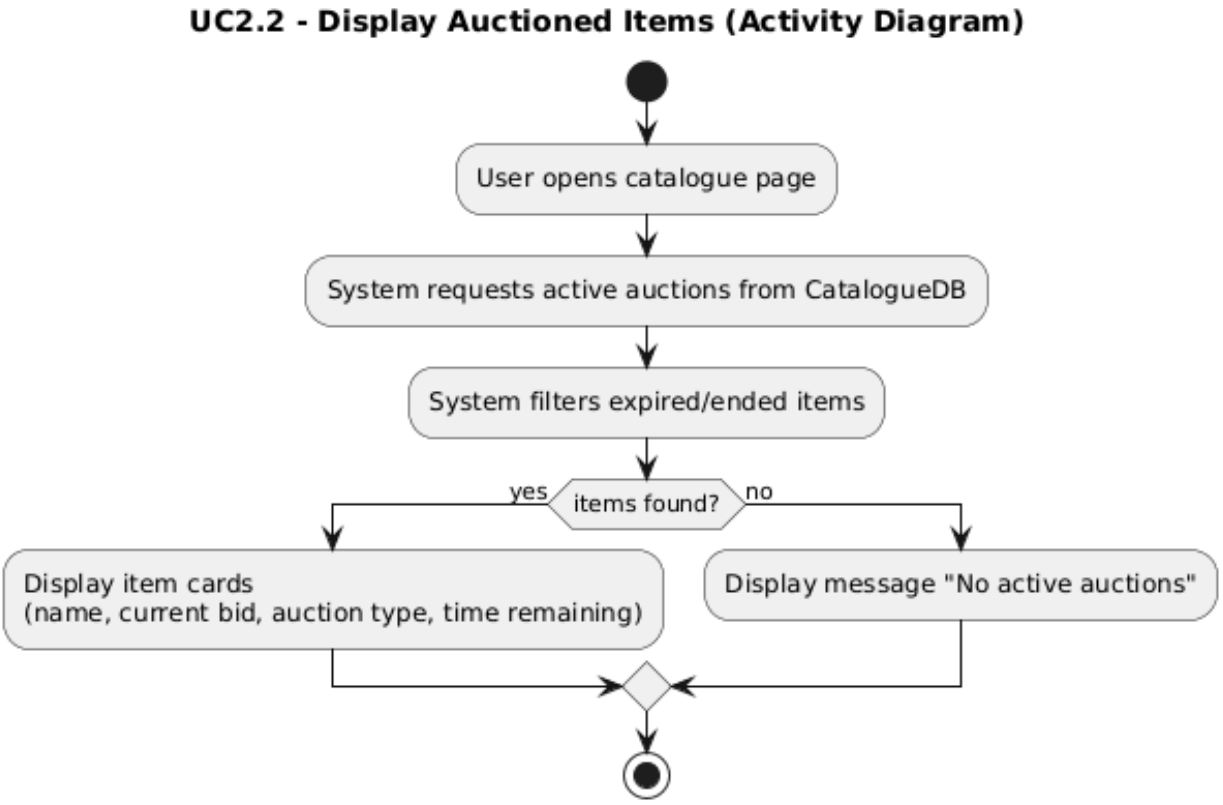
This diagram models how a user selects exactly one item to bid on. After the system displays the auction item list, the user selects an item using a radio button. A decision checks if exactly one item is selected. If not, the user is prompted to select exactly one item; otherwise the user proceeds by clicking BID, and the system loads the item details and displays the bid page.



This activity diagram captures the system behavior when an auction ends. After the timer expires, the system determines the winner and displays the “Auction Ended / Pay Now” page. A decision checks whether the current user is the winner; non-winners receive a failure notice and cannot proceed. Winners can optionally select expedited shipment, which updates the shipping cost, then click “Pay Now” to proceed to the payment page.



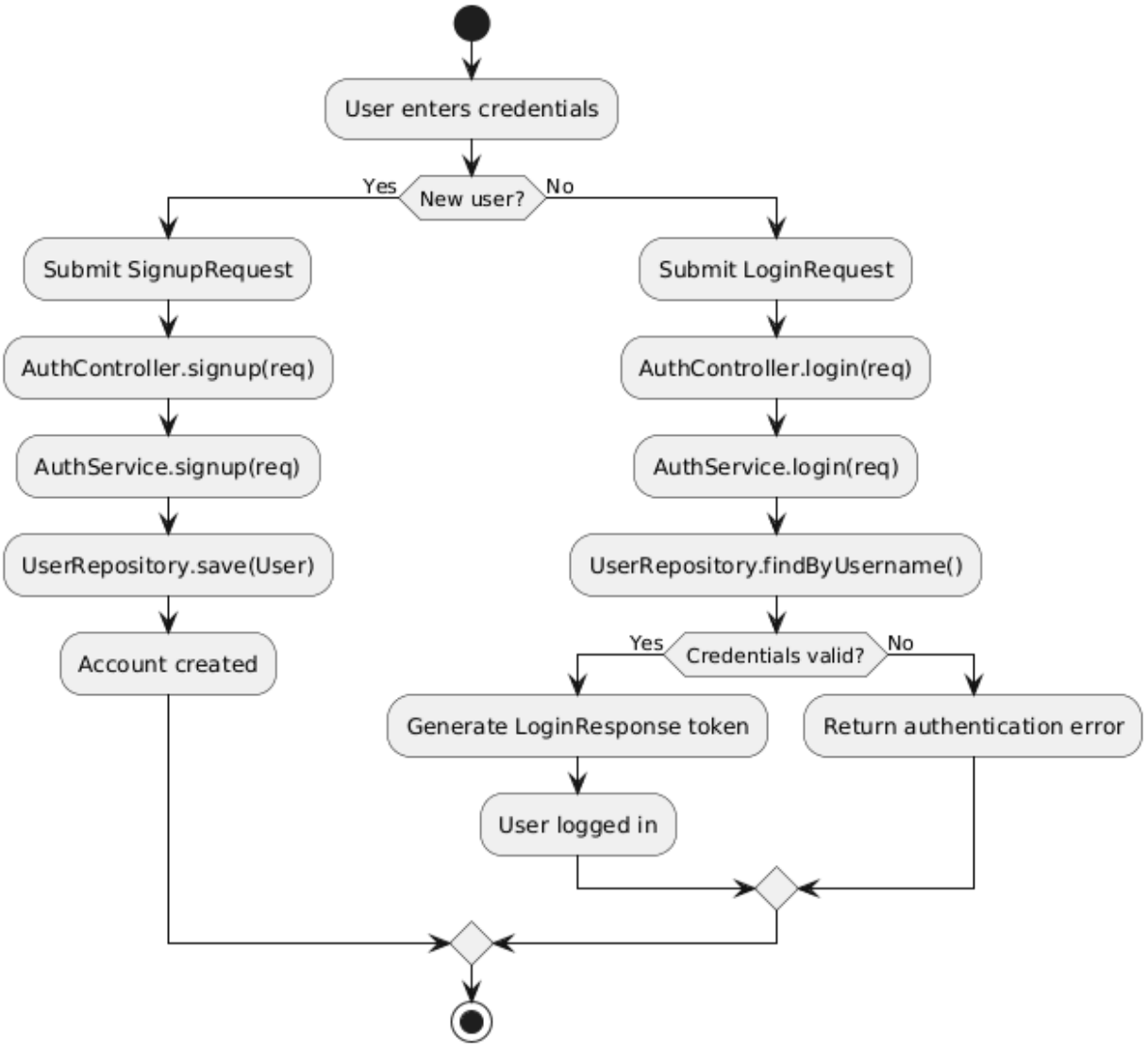
This diagram shows what happens after payment is confirmed. The system receives confirmation, generates a receipt containing the total paid, retrieves shipment timing , and displays both the receipt and shipping information. The user may then return to the main page.



This activity diagram shows how the system displays auction items after the user opens the catalogue page. The system requests active auctions, filters expired/ended items, and then branches based on whether items are found. If auctions exist, item cards are displayed; otherwise the user receives a “No active auctions” message.

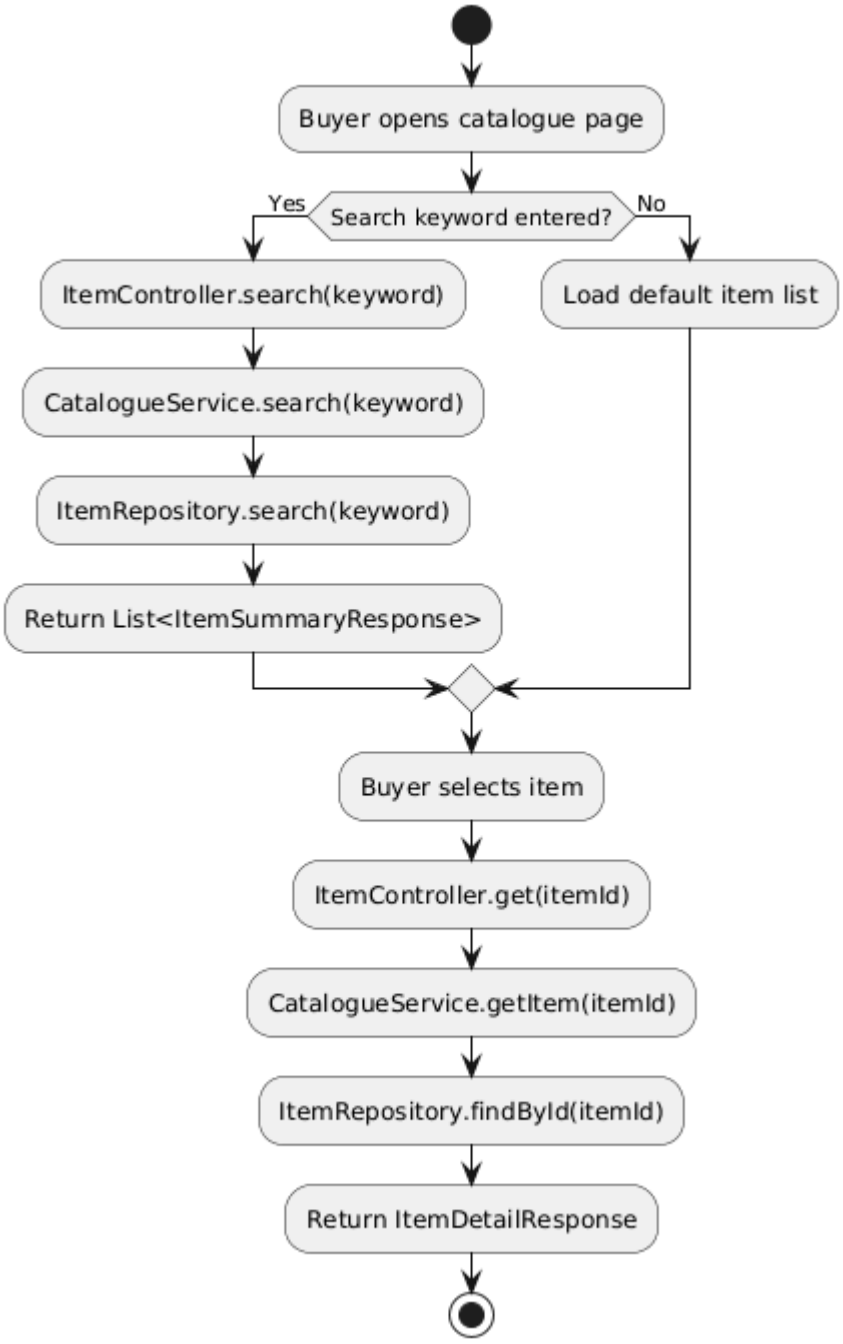
4 (Appended)Activity Diagrams

UC1: Sign-up and Login



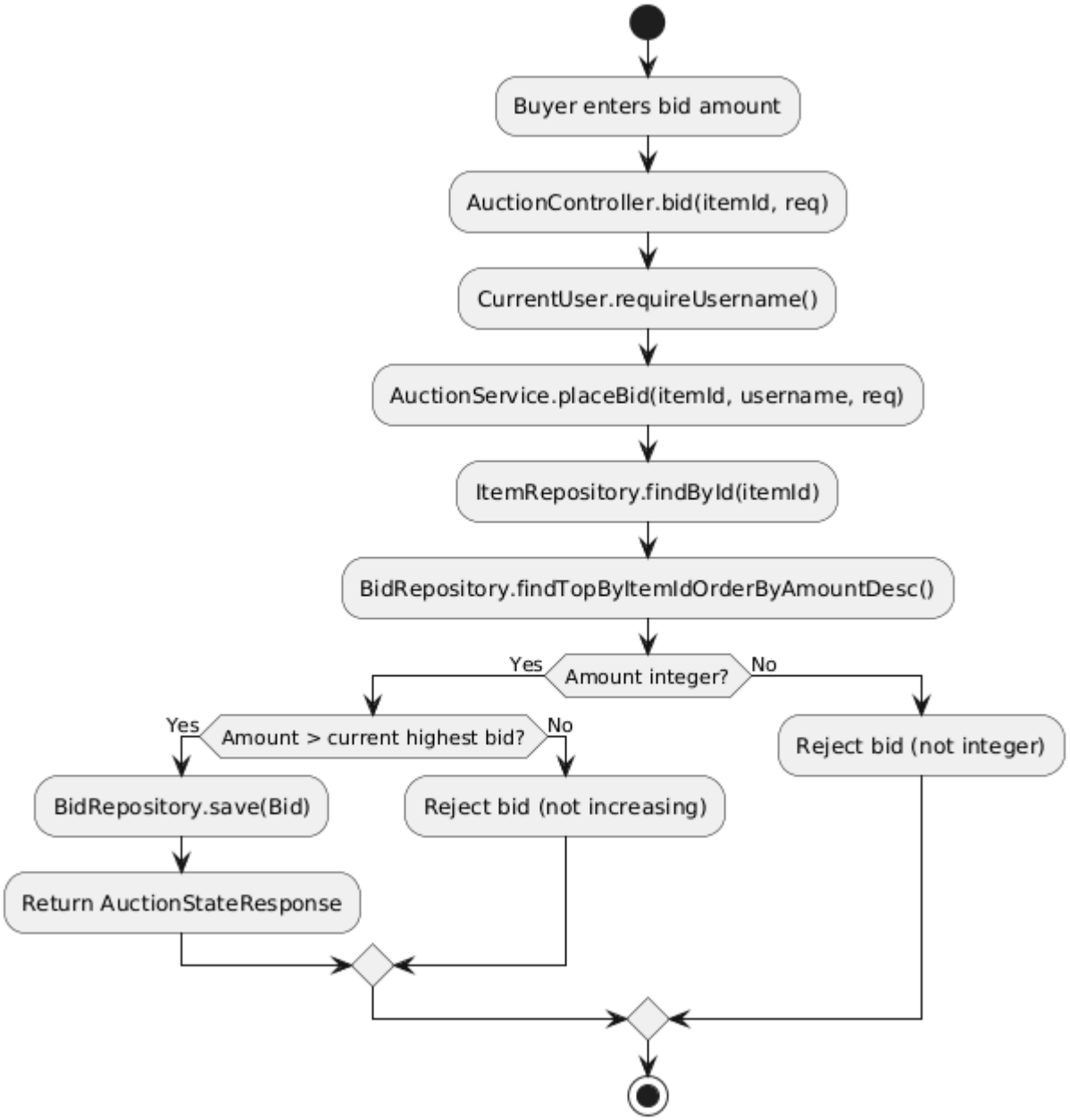
This activity diagram shows how the system handles user authentication for both new and returning users. The user submits credentials through the interface, and the system branches depending on whether the user is signing up or logging in. For sign-up, the system creates a new account in the database. For login, the system verifies the credentials and either grants access by generating a session token or returns an authentication error.

UC2: Browse and Search Catalogue



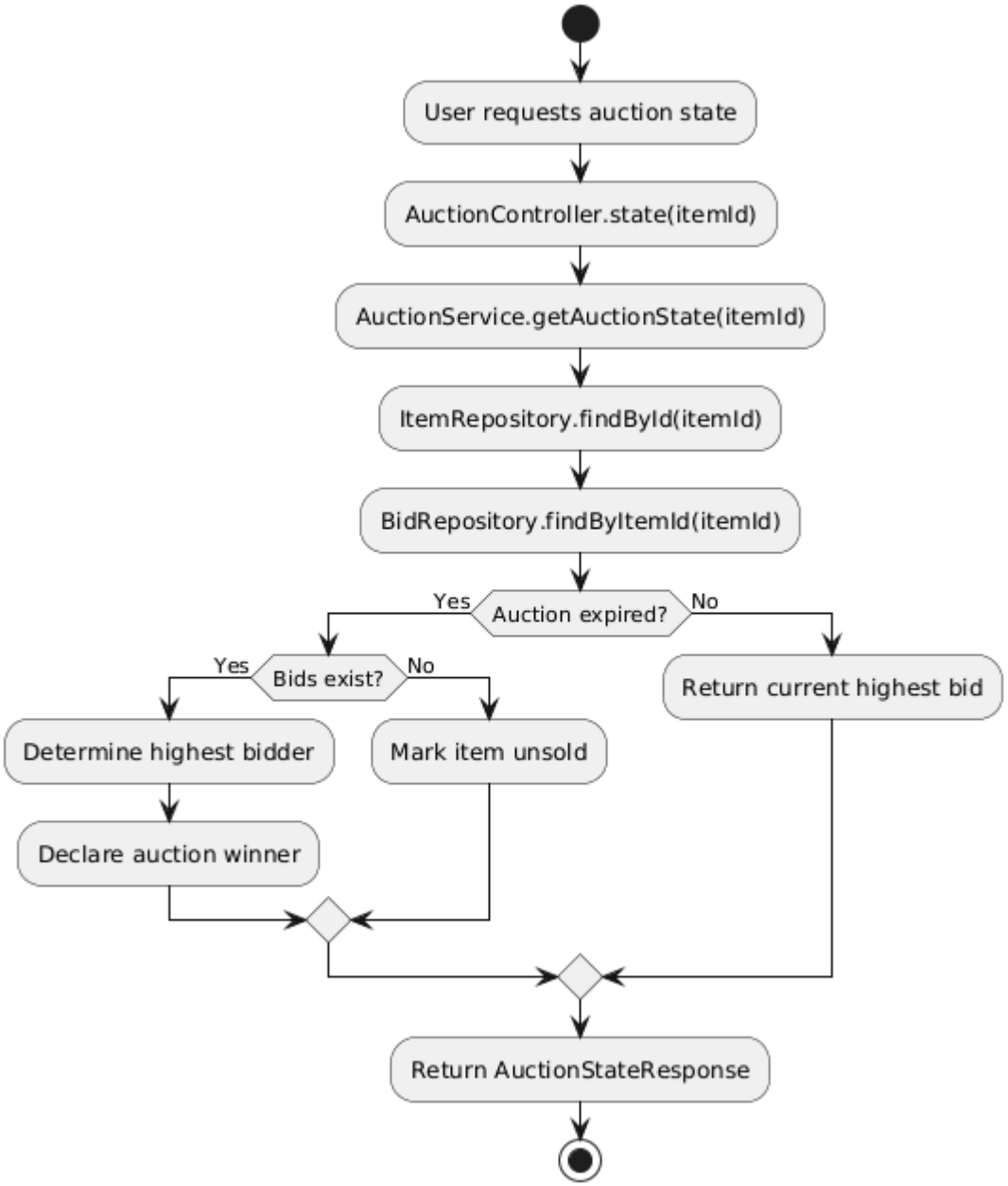
This activity diagram shows how the system displays auction items when a buyer browses the catalogue. The system first checks whether the user entered a search keyword. If a keyword is provided, the system searches the catalogue and retrieves matching items from the database. If no keyword is entered, the system loads the default list of items. The user can then select an item to view its detailed information.

UC3: Placing and Validating Bids



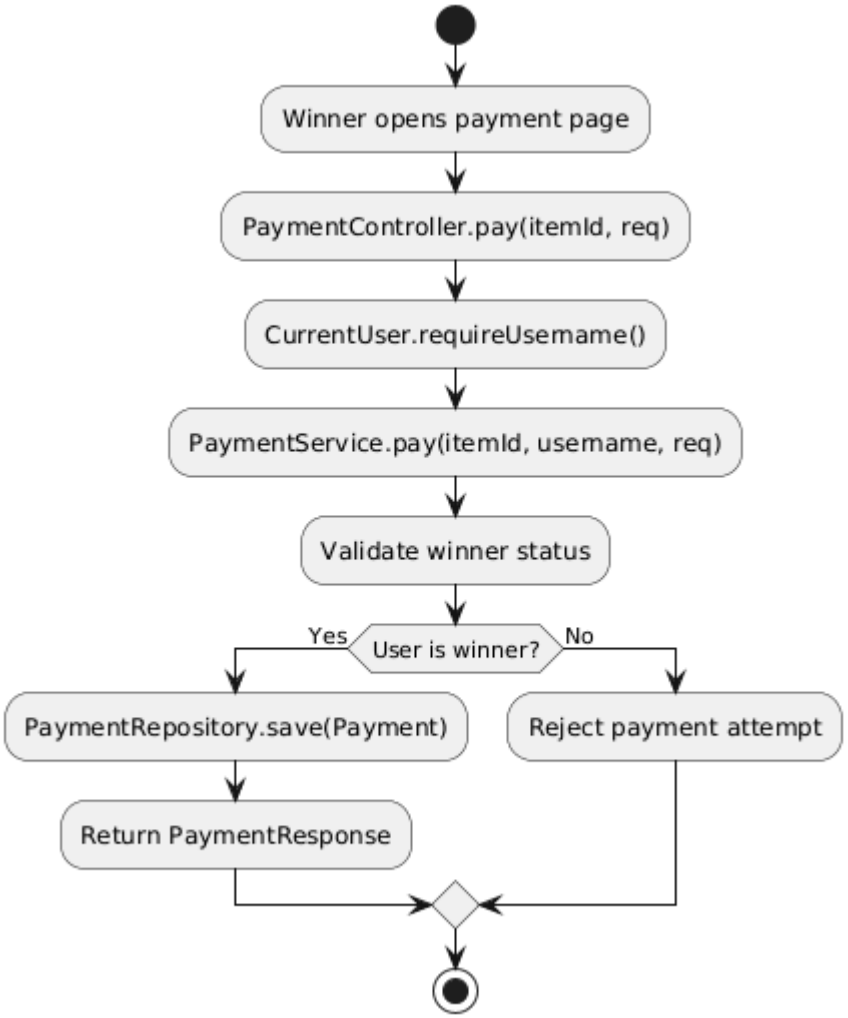
This activity diagram illustrates how the system processes a buyer's bid on an auction item. After the buyer submits a bid amount, the system retrieves the current highest bid and validates the new bid. The bid must be an integer and strictly greater than the existing highest bid. If both conditions are satisfied, the bid is saved and the auction state is updated. Otherwise, the system rejects the bid and informs the user of the validation error.

UC4: Auction End Behaviour



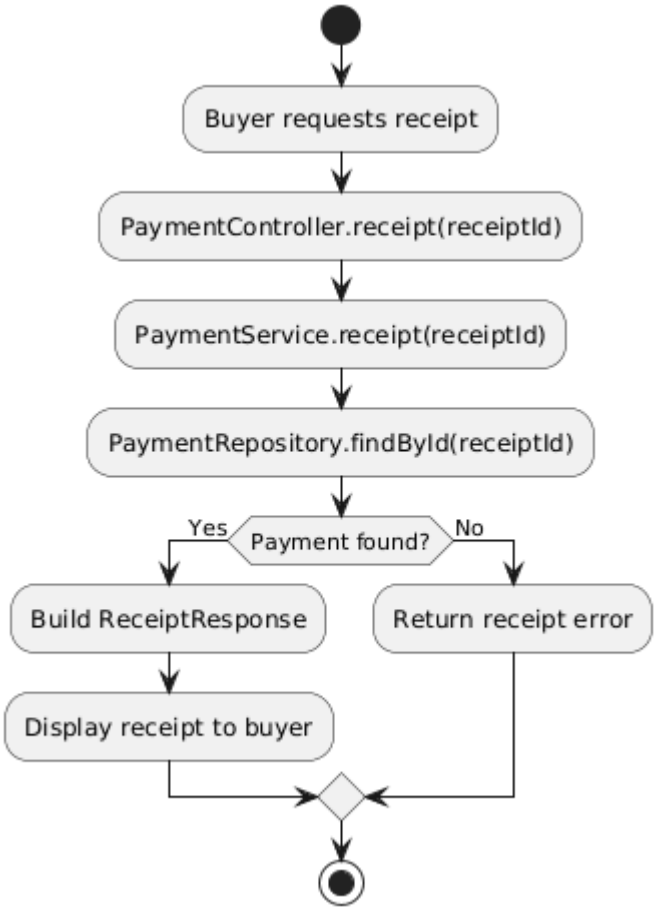
This activity diagram shows how the system determines the outcome of an auction when its state is requested. The system retrieves the item and its associated bids, then checks whether the auction has expired. If the auction has ended and bids exist, the system determines the highest bidder and declares the winner. If no bids exist, the item is marked as unsold. If the auction is still active, the current highest bid is returned.

UC5: Payment



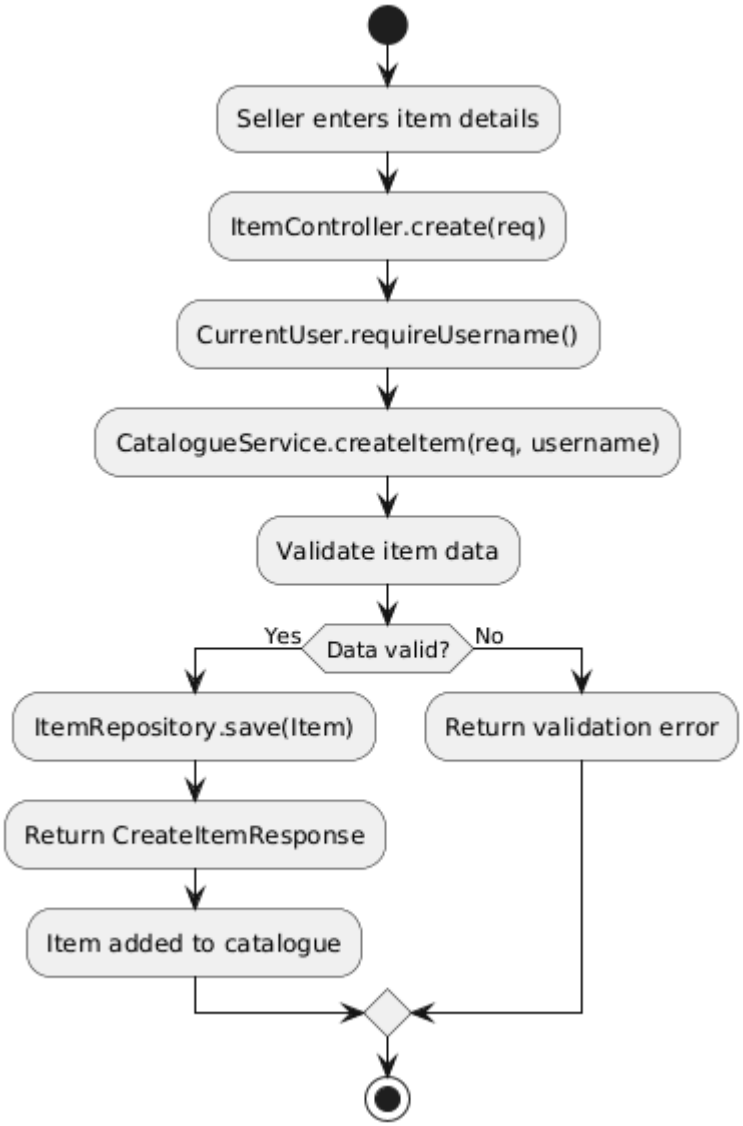
This activity diagram shows how the system processes payment for a completed auction. When the winning bidder submits payment information, the system verifies the user's identity and confirms that they are the winner of the auction. If the user is authorized, the payment is recorded in the database and a payment response is returned. If the user is not the winner, the system rejects the payment attempt.

UC6: Receipt and Shipping



This activity diagram illustrates how the system retrieves and displays a payment receipt. When the buyer requests the receipt, the system searches for the corresponding payment record. If the record exists, the system generates a receipt response containing the transaction details and displays it to the user. If the payment record cannot be found, the system returns an error message.

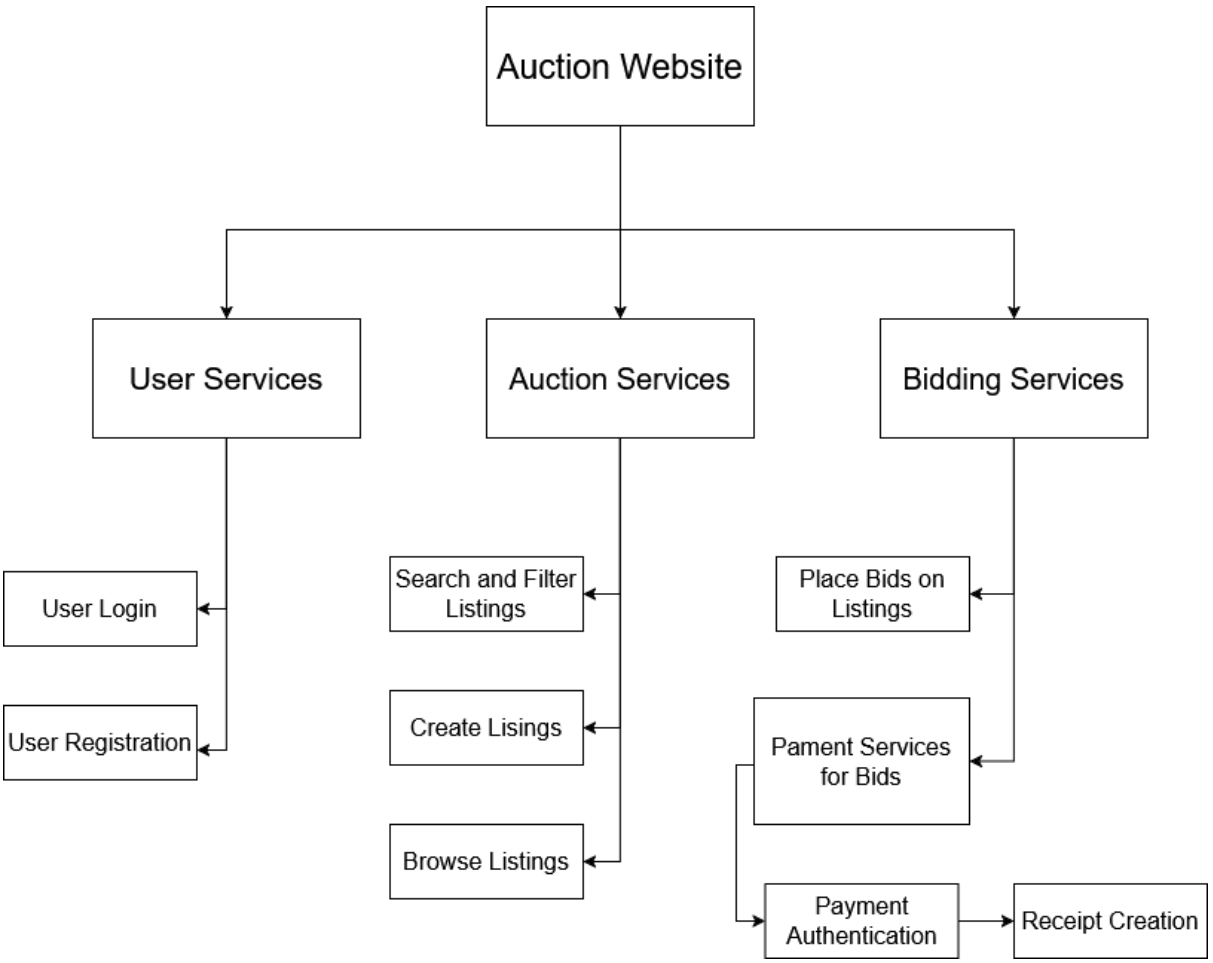
UC7: Creating An Auction Listing



This activity diagram shows how a seller uploads a new item to the auction catalogue. The seller enters the item details, which are sent to the system for processing. The system verifies the seller's identity and validates the item information. If the data is valid, the item is saved in the database and added to the catalogue. If validation fails, the system returns an error message and the item is not created.

5 Architecture

5.1 Decomposition Diagram



5.2 Services Table

Services			
Service Name	**Description**	**Exposed Interface Name**	**Interface Description**
M1	User login and registration	M1:I1 M1:I2	M1:I1 Handles user sign in M1:I2 Handles user registration
M2	Manages payment authentication and receipt handling	M2:I3 M2:I4	M2:I3 Handles processing user payment information

			M2:I4 Handles generating receipt for user purchase
M3	Handles auction browsing, search and filtering functionality	M3:I5	M3:I5 Manages searching, filtering and browsing of auction listings
M4	Handles placing bids on auction items	M4:I6	M4:I6 Handles bidding operations
M5	Handles creation of auction listings	M5:I7	M5:I7 Manages the creation of auction listings

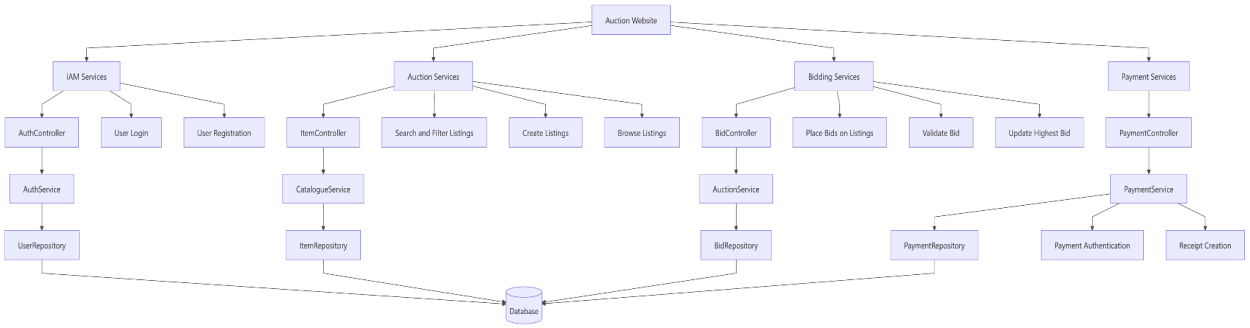
5.3 Interfaces Table

Interfaces		
Interface Name	**Operations**	**Operation Descriptions**
M1:I1	<boolean> Op1(credentials) used by M1	M1:I1:Op1: handles signing into the website with user credentials
M1:I2	<Account> Op2(String userEmail, String userPassword)	M1:I2:Op2: registration interface that handles account creation
M2:I3	<boolean> Op3 (enterPaymentDetails cardInfo) <boolean> Op3 (validatePaymentDetails cardInfo)	M2:I3:Op3: enterPaymentDetails handles submission of payment information M2:I3:Op3: validatePaymentDetails handles processing and authenticating payment
M2:I4	<String> Op4 (String genReceipt) <String> Op4 (String displayReceipt)	M2:I4:Op4: genReceipt handles the creation of the receipt for purchase M2:I4:Op4: displayReceipt displays a viewable and savable copy of the order receipt

M3:I5	<String> Op5 (String searchItems) <String> Op5 (String itemList)	M3:I5:Op5:searchItems handles querying the website database for auction items M3:I5:Op5: itemList shows results from search query if there are any
M4:I6	<boolean> Op6 (int placeBid)	M4:I6:Op6:placeBid handles bidding on auction listings
M5:I7	<listing> Op7 (String listingName, String listingInfo, int startingBid)	M5:I7:Op7: handles the creation of an auction listing

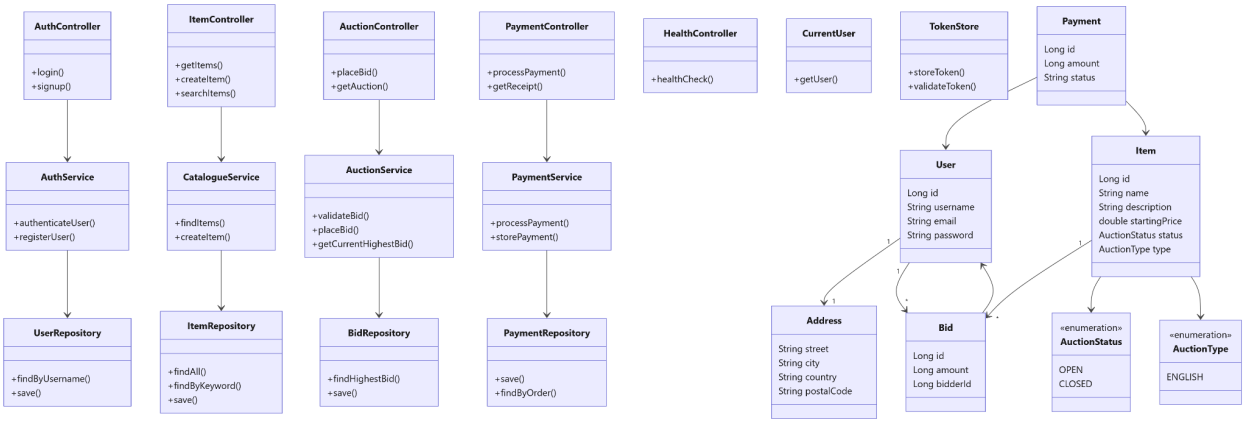
5 (Appended) Architecture

5.1 (Appended) Decomposition Diagram



For a better image of the system architecture diagram click here.

Class Diagram:



For a better image of the class diagram click here.

5.2 (Appended) Services Table

Service Name	Description	Exposed Interface	Interface Description
Authentication Service	Handles user authentication and account creation including login and registration.	M1:I1 – Login Interface M1:I2 – Registration Interface	M1:I1 Handles user sign in M1:I2 Handles user registration
Catalogue Service	Handles browsing, searching, and viewing auction listings as well as creating new listings.	M3:I5 – Listing Browsing Interface	M3:I5 Manages searching, filtering and browsing of auction listings
Auction Service	Manages auction bidding logic and auction state information.	M4:I6 – Bidding Interface	M4:I6 Handles bidding operations
Payment Service	Handles payment processing for auction winners. Generates and displays purchase receipts after payment is completed.	M2:I3 – Payment Interface M2:I4 – Receipt Interface	M2:I3 Handles processing user payment information M2:I4 Handles generating receipt for user purchase

5.3 (Appended) Interfaces Table

Interfaces		
Interface Name	**Operations**	**Operation Descriptions**
Login Interface	showLoginForm() enterCredentials(LoginRequest req) login(LoginRequest req) findByUsername(String username) validatePassword(String password)	Displays the login page to the user. Allows the user to enter login credentials. Sends the login request to the authentication controller.

	LoginResponse displayHomePage() displayError(String message)	Retrieves the user record from the database. Validates the user password. Contains the result of the login attempt. Redirects the user to the system homepage after successful login. Displays an error message if authentication fails.
M1:I2	clickSignUp() showSignUpForm() enterUserDetails(SignupRequest req) signup(SignupRequest req) validateSignup(SignupRequest req) save(User user) SignupResponse displayError(String message) showLoginForm()	User selects the option to register a new account. Displays the sign-up form. User enters registration information. Sends registration request to the authentication controller. Validates the entered registration information. Stores the new user in the database. Contains the result of the registration process. Displays error if registration fails. Redirects user to login page after registration.
M2:I3	enterPaymentDetails(PaymentRequest req) pay(Long itemId, PaymentRequest req) validatePayment(PaymentRequest req)	User enters payment information. Sends payment request to the payment controller.

	save(Payment payment) PaymentResponse displayError(String message)	Verifies payment information. Stores payment record in the database. Returns result of payment transaction. Displays payment error message
M2:I4	viewReceipt(Long receiptId) receipt(Long receiptId) findById(Long receiptId) ReceiptResponse displayReceipt()	User requests receipt for a completed purchase. Retrieves receipt information from payment service. Retrieves payment record associated with the receipt. Contains receipt information. Displays receipt to the user.
M3:I5	enterSearch(String keyword) search(String keyword) findItems(String keyword) queryItems(String keyword) List<ItemSummaryResponse> displayResults(List<ItemSummaryRespon se>) displayNoResults() get(Long itemId) create(CreateItemRequest req) save(Item item)	User enters a search keyword. Sends search request to the item controller. Catalogue service processes the search query. Retrieves matching items from the database. Returns list of items matching the search criteria. Displays matching auction listings. Displays message if no items are found. Retrieves detailed information for a selected item.

		Allows sellers to upload a new auction item
M4:I6	enterBid(BidRequest req) bid(Long itemId, BidRequest req) placeBid(Long itemId, String username, BidRequest req) findTopByItemIdOrderByAmountDesc(Long itemId) validateBid(BidRequest req) save(Bid bid) AuctionStateResponse state(Long itemId) displayUpdatedBid() displayError(String message)	User enters a bid amount. Sends bid request to the auction controller. Processes bid logic in auction service. Retrieves the highest bid for the item. Checks whether bid is valid and higher than current bid. Stores the bid in the database. Returns updated auction state. Retrieves auction status. Displays updated highest bid. Displays bid rejection message.

6 Activities Plan

6.1 Product Backlog for Auction System

#	Requirements/Feature	Priority	Status
1	Login and Registration Page	High	In Progress
2	Profile Management page	High	In Progress
3	Create Listing page	High	In Progress
4	View Listings page	Medium	In Progress

5	Browsing/Searching Listings	Medium	In Progress
6	Placing Bids on Listings	High	In Progress
7	Payment page	High	In Progress
8	Notifications	Medium	In Progress
9	Watchlist Page	Low	In Progress
10	Shipping Page	High	In Progress
11	Auction History	Low	In Progress

6.1 (Appended) Product Backlog for Auction System

#	Requirements/Feature	Priority	Status
1	Login and Registration Page	High	Completed
2	Profile Management page	High	Completed
3	Create Listing page	High	Completed
4	View Listings page	Medium	Completed
5	Browsing/Searching Listings	Medium	Completed
6	Placing Bids on Listings	High	Completed
7	Payment page	High	Completed
8	Notifications	Medium	In Progress
9	Watchlist Page	Low	In Progress
10	Shipping Message	High	Completed
11	Auction History	Low	In Progress
12	Auction End Behaviour	High	Completed
13	Receipt Message	Medium	Completed
14	Admin Dashboard	High	Completed
15	Role-Based Access Control (RBAC)	High	Completed
16	Docker Containerization	High	Completed
17	React TypeScript Frontend	High	Completed

6.2 Deliverable 1 Sprint Backlog

#	Section of Software Design Document	Priority	Status
1	Title Page	High	Completed
2	Introduction	High	Completed
3	Major Design Decisions	High	Completed
4	Sequence Diagrams	High	Completed
5	Activity Diagrams	High	Completed
6	Architecture	High	Completed
7	Activities Plan	High	Completed
8	Group Meeting Logs	High	Completed
9	Test Driven Developments	High	Completed
10	AI Acknowledgement	High	Completed

6.3 Deliverable 2 Sprint Backlog

Task	Priority	Status	Feature
UC 1 Implementation	High	In Progress	1-2
UC 2 Implementation	High	In Progress	3-5
UC 3 Implementation	High	In Progress	6
UC 4 Implementation	High	In Progress	7
UC 5 Implementation	High	In Progress	10
UC 6 Implementation	High	In Progress	8,9,11

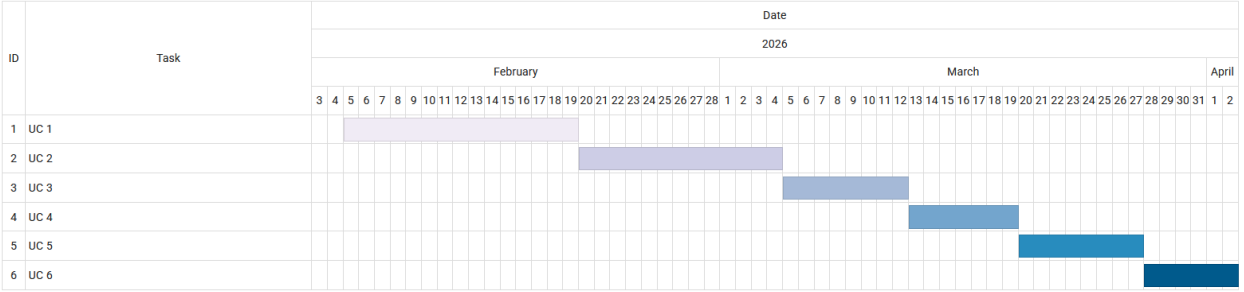
6.3 (Appended) Deliverable 2 Sprint Backlog

Task	Priority	Status	Feature
UC 1 Implementation	High	Completed	1
UC 2 Implementation	High	Completed	5
UC 3 Implementation	High	Completed	6
UC 4 Implementation	High	Completed	12
UC 5 Implementation	High	Completed	7
UC 6 Implementation	High	Completed	10, 13
UC 7 Implementation	High	Completed	3

6.4 (Appended) Deliverable 3 Sprint Backlog

Task	Priority	Status	Feature and Requirement
Full System Functional and Tested	High	Completed	UC1–UC7
React Frontend Implemented	High	Completed	All pages
Role-Based Access Control (RBAC)	High	Completed	Admin feature
Docker Containerization	High	Completed	Deployment
Admin Postman Tests	High	Completed	Security Testing
Design Document Updated	High	Completed	Documentation

6.5 Gantt Diagram



7 (Appended) Group Meeting Logs

Meeting 1 – Project Kickoff and Task Allocation
The first group meeting was held on Tuesday, January 27, where we reviewed the course project and Deliverable 1 requirements in detail. During this meeting, we discussed the scope of the deliverable, including the required UML diagrams, architecture specification, activity plan, and test cases. The group aligned on the overall approach and identified the key activities needed to complete Deliverable 1. Tasks were then assigned to each group member based on the required sections of the deliverable. Following this meeting, the team continued to collaborate asynchronously through the Discord group channel by sharing progress updates, providing feedback, and coordinating work.

Meeting 2 – Progress Review and Internal Deadline Check
The second meeting was held on Wednesday, February 4, which served as the group's internal soft deadline for Deliverable 1. During this meeting, members presented their completed or in-progress sections, and the group reviewed the work collectively for completeness and consistency. The team also cross-checked the deliverable against the grading criteria posted on eClass to ensure alignment with the evaluation requirements. Feedback was shared, and final refinements were discussed to prepare the deliverable for submission.

7 (Appended) Group Meeting Logs

Meeting 3 – Deliverable 2 Discussion and Task Assignment
The third meeting was held on Friday, February 20, where the group discussed the second part of the project and what was required for Deliverable 2. During this meeting the group organized what needed to be done, how it would be done and who would be doing it. The group discussed and aligned the overall approach we would be taking to complete the project as well as setting up an internal deadline for all tasks to be completed by. Afterwards, group members were assigned tasks to complete. Following this, the group collaborated asynchronously through the team discord server.

Meeting 4 – Deliverable 2 Progress Report and Internal Deadline
The fourth meeting was held on Monday, March 2, where the group met up in accordance with the internal deadline set during the first meeting. Here we checked in, presented the work done and cross checked each other's work. The team then made final adjustments and alterations based on feedback given to each other and prepared for submission.

Meeting 5 – The fifth meeting was held in early March where the group discussed the requirements for Deliverable 3. The group reviewed the full system functionality requirements and aligned on the React TypeScript frontend as the primary remaining implementation task. Role-Based Access Control was confirmed as the distinguishable feature. Tasks were divided as follows: Mahmoud and Yazan handled the frontend implementation. Amanullah handled Docker containerization, Lama handled the admin Postman test scripts, and Tristan handled the design document updates. An internal deadline was set and the group continued to collaborate asynchronously through the team Discord server.

Meeting 6 – The sixth meeting was held on March 26, 2026 where the group conducted a final end-to-end review of the complete system. The Docker Compose setup was verified, the Postman happy path and negative tests were run successfully against the containerized backend, and the full frontend flow was demonstrated from signup through receipt generation. Final adjustments were made to the README, design document, and submission package, and the group confirmed all Deliverable 3 requirements were met prior to submission.

8 (Appended) Test-driven development

Note: JUnit tests were prepared as part of the test planning process. The professor confirmed that Postman collections satisfy the testing requirement for this project

9 (Appended) Test plan Summary

The tests are split between running using Postman Collections to test the End-To-End Flow (Happy Path in the README) and to test the Robustness (Negative Tests in the README) test cases in the tests file. The Happy Path covers UC1-UC7 which encompasses Sign Up and In, Browse and Searching the Catalogue, whether bids are valid, Auction End Behaviour, Payment, Receipt and Shipping Messages, and Creating a Catalogue Item. The Negative Path tests cover errors that can be expected when using the system as normal such as duplicate usernames while signing up, invalid login details, invalid bid amounts, bidding after an auction has ended, a non-winning attempting to pay and missing payment fields. The tests in the test case file cover the same things as previously stated just in the form of test cases as opposed to using Postman.

9.1 (Appended) Postman and JUnit Test Cases (See GitHub)

For Deliverable 3, the test plan was extended to include security and role-based access control testing. Six additional Postman tests were added to verify that administrative endpoints are properly protected. These tests confirm that GET /api/admin/stats, DELETE /api/admin/items/{itemId}, and POST /api/admin/items/{itemId}/end all return 200 when accessed with an admin token and 403 when accessed with a regular user token. These tests are included in the JavaBrew D3 Postman collection under the Admin Tests folder.

For what the Postman tests cover see the README on GitHub here.

The following changes were made in the TDD with respect to the JUnit test cases:

- 8.2 tests are added to 8.1 to test for UC1 (Sign Up & Log In), 8.2 is now for UC3 (Placing and Validating Bids)
- 8.3 is now representing UC4 (Auction End Behaviour) instead of Data Entry
- 8.4 is now representing UC5 (Payment) instead of Reporting
- 8.5 is now representing UC6 (Receipt and Shipping) instead of User Profile

For the JUnit tests see the GitHub here.

9.2 (Appended) JMeter Performance Test Results

Performance testing was also conducted using Apache JMeter on a locally hosted Spring Boot application (localhost:8080). Two endpoints were evaluated: GET /api/items and POST /api/auth/login. Tests were performed with 10, 50, 100, and 200 concurrent users, with each user sending one request. Response time metrics, including average, median, and 90th/95th percentiles, were collected using the Aggregate Report.

GET /api/items

Num of Users	Average (ms)	Median (ms)	90% Line (ms)	95% Line (ms)	Error %
10	6	6	9	9	0.00
50	7	6	8	9	0.00
100	4	4	6	6	0.00
200	4	4	5	5	0.00

POST /api/auth/login

Num of Users	Average (ms)	Median (ms)	90% Line (ms)	95% Line (ms)	Error %
10	128	122	136	136	0.00
50	119	118	129	131	0.00
100	126	125	138	147	0.00
200	123	123	131	137	0.00

The /api/items endpoint demonstrated consistently low response times, ranging from 4 to 7 ms across all load levels, indicating efficient handling of read operations and strong scalability. In contrast, the /api/auth/login endpoint exhibited higher response times, averaging approximately 119–128 ms due to additional processing such as password hashing and database validation. Despite this, response times remained stable as concurrency increased, suggesting no significant performance degradation.

System load was normalized using the formula v/v0, where v0=500. The tested loads correspond to normalized values of 0.02, 0.10, 0.20, and 0.40 for 10, 50, 100, and 200 users respectively. These results

indicate that the system maintains reliable performance up to 40% of its baseline capacity, with no major bottlenecks observed.

For the JMeter Test results see the GitHub here.

10 (Appended) Distinguishable Feature

10.1 Role-Based Access Control

A role-based access control system was implemented as the distinguishable feature for Deliverable 3. The system supports two roles: USER and ADMIN. All accounts created through the standard signup flow are assigned the USER role by default. Admin accounts are seeded directly into the H2 database on application startup through AdminSeeder.java.

Users are permitted to browse the catalogue, view item details, place bids on active auctions, complete payment for auctions they have won, and view receipts. Admins are additionally permitted to view platform statistics including total user count, total item count, active auction count, and total revenue. Admins can also force-end any active auction immediately and delete any item listing from the catalogue regardless of auction status.

Access control is enforced at two levels. On the backend, the AdminController validates the authenticated user's role before processing any administrative request. If the user does not hold the ADMIN role, the request is rejected with a 403 Forbidden response. On the frontend, admin-only UI elements including the Admin navigation tab, per-item delete buttons, and force-end auction controls are conditionally rendered based on the role field returned by the /api/auth/me endpoint. This prevents admin controls from appearing in the interface for regular users while still enforcing authorization independently on the server side.

10.2 Admin AI Analytics Feature

An AI-powered admin assistant was implemented as an additional distinguishable feature for Deliverable 3. The assistant allows administrators to ask natural-language questions about marketplace activity and receive summarized responses directly within the admin dashboard. Supported queries include determining the number of active auctions, identifying items with no bids, viewing auctions with the highest current bids, and summarizing recent bidding activity and overall marketplace trends.

The feature is implemented through a new backend endpoint located at /api/admin/ai/chat. Several analytics helper methods were added to support the assistant, including methods for retrieving platform statistics, filtering auctions by status, identifying top auctions by current bid, finding items with no bids, and generating bid activity summaries. On the frontend, a dedicated AI chat panel was added to the admin dashboard, along with response typing support for the new endpoint.

Security for the AI assistant is enforced in both the backend and frontend. The OpenAI API key is not stored in the frontend and is not hardcoded in any source files. Instead, the backend reads the key from the OPENAI_API_KEY environment variable and reads the model name from OPENAI_MODEL. In

addition, access to the AI assistant is restricted exclusively to users with the ADMIN role. Non-admin users cannot access the AI endpoint or view the AI panel in the frontend interface.

11 (Appended) Frontend Implementation

A React TypeScript frontend was implemented for Deliverable 3 to provide a complete and fully functional user interface for the auction system. The frontend communicates with the Spring Boot backend exclusively through REST API calls using Axios. React Router v6 is used for client-side navigation between pages. Styling is implemented using plain CSS with a consistent card-based layout.

The following pages were implemented. The Login page allows returning users to authenticate using their username and password. The Signup page collects all required user information including first name, last name, username, password, and full shipping address. The Catalogue page displays all active auction listings and supports keyword-based search. The Item Detail page displays full item information, the current highest bid, the highest bidder, a live countdown timer that updates every second, and a bid submission form for active auctions. When an auction ends, the item detail page shows a winner notification and Pay Now button for the winning bidder, and a failure notice for all other users. The Create Item page allows authenticated users to create new auction listings by entering item details, starting bid, duration, and shipping information. The Payment page displays an order summary with dynamic shipping cost calculation, an expedited shipping toggle, and a credit card entry form. The Receipt page displays the receipt ID, total amount paid, and estimated shipping information after a successful payment. The Admin Dashboard is accessible only to admin users and displays platform statistics and provides controls for force-ending auctions and deleting items.

Authentication state is managed through a React context that stores the bearer token in localStorage and attaches it to every outgoing Axios request via a default header. On application load, the context checks for a stored token and calls /api/auth/me to restore the user session. Protected routes redirect unauthenticated users to the login page. Admin-only routes redirect non-admin users to the catalogue.

12 (Appended) Docker Deployment

The backend and frontend are each packaged as independent Docker containers. Docker Compose is used to orchestrate both containers, allowing the complete application stack to be launched with a single command from the project root directory.

```
docker-compose up --build
```

The backend container is built using the Eclipse Temurin JDK 17 base image. The Maven wrapper is used inside the container to compile the project and package it as an executable JAR, which is then launched using the java -jar command. The backend is exposed on port 8080.

The frontend container is built using the Node 20 Alpine base image. Dependencies are installed using npm install and the Vite development server is started with host binding set to 0.0.0.0 to allow external access from outside the container. The frontend is exposed on port 5173. The VITE_API_URL environment variable is passed to the frontend container to configure the backend API base URL.

The deployment architecture is as follows:

Browser
↓
React Frontend — Docker container (port 5173)
↓
Spring Boot Backend — Docker container (port 8080)
↓
H2 In-Memory Database (embedded in backend container)

Note that the H2 in-memory database resets each time the backend container is restarted. The Postman happy path collection should be run after startup to seed initial data for demonstration purposes. Admin credentials are seeded automatically on startup by AdminSeeder.java and do not require manual setup.

13 (Appended) AI Advantages and Disadvantages

Note: This list is with regard to the creation of the Auction System and completion of the Software Design Document.

Advantages:

- Improved productivity by generating examples and templates that could be adapted into the design document
- Helped organize the structure of the report and improve clarity, formatting, and readability
- Assisted with debugging by suggesting possible causes of errors in backend, frontend, and Docker configurations
- Provided guidance on REST API design, RBAC implementation, Postman tests, and JMeter testing strategies

Disadvantages:

- AI-generated code was not always correct and contained syntax errors, outdated methods, or logic that did not fit the project requirements
- AI sometimes suggested unnecessary complexity or technologies that were not appropriate for the scope of the project
- Certain outputs lacked project-specific context, requiring the team to manually adjust names, endpoints, class structures, and workflows
- Overreliance on AI could reduce understanding of the underlying concepts if suggestions were accepted without verification
- All generated content had to be reviewed, tested, and validated to ensure correctness and consistency with the implemented system

14 (Appended) AI Acknowledgement

AI tools were used to support productivity during this project, primarily for brainstorming, documentation, and refining written content. All AI-assisted artifacts were reviewed, understood, critiqued, and modified by the team. The final design decisions, implementations, and maintenance of the project remain the responsibility of the authors.

(Appended)

AI tools were used throughout this project to support productivity, improve documentation quality, and assist with problem-solving. The primary tool used was ChatGPT.

AI assistance was used in the following ways:

- Generating initial drafts and refining sections of the Software Design Document, including explanations for architecture, design patterns, and testing strategies
- Assisting in structuring and organizing report sections to improve clarity and readability
- Providing guidance on REST API design, Docker configuration, and frontend-backend integration
- Helping generate example test scenarios and improving descriptions of performance and testing reports

All AI-generated content was carefully reviewed, validated, and modified by the team before inclusion. The team ensured full understanding of all generated material and verified its correctness against the implemented system.

No AI-generated code was directly copied without review; all implementation decisions and final code were written, tested, and maintained by the team. AI tools were used strictly as a support mechanism and not as a substitute for understanding or development effort.

15 (Appended) Deviations from Original Design

The implemented system preserved the overall modular and service-oriented direction proposed in Deliverable 1, but several refinements were made during implementation to better align the design with the actual Spring Boot backend.

First, the abstract service naming used in Deliverable 1, such as M1, M2, and M3, was replaced with concrete implementation-oriented components including AuthController, AuthService, ItemController, CatalogueService, AuctionService, PaymentService, and their associated repositories. This change improved clarity and created direct traceability between the design document and the implemented codebase.

Second, the implementation adopted a layered Spring Boot architecture consisting of Controllers, Services, Repositories, DTOs, and Entities. While the original design described a modular service-based system, the implemented architecture refined this into a concrete layered structure that better supports maintainability, separation of concerns, and testing.

Third, token-based stateless authentication was introduced in the implementation to support protected API access. This was a refinement of the original authentication design and was chosen to better support REST principles and future microservice deployment.

Finally, the architecture tables, decomposition diagrams, and sequence diagrams were updated to reflect the actual implementation details and naming conventions used in the backend. These changes do not

represent a change in the core system goals, but rather an implementation-level refinement of the original design.

16 (Appended) Amendments to the Original Document

- Changed the decomposition diagram in 5.1 of Architecture to reflect the current system Architecture including service diagrams
- Fixed the Services Table (5.2) and the Interfaces Table (5.3) to better reflect the names of variables and design changes made in the implementation
- Appended the group meeting log (section 7) with what was discussed in the meetings
- Added and updated the test plan summary (section 9) to reflect what the Postman and test cases do and what they test in the implementation
- Added the "Deviations from Original Design" section (section 11) that discusses and lists any changes made to the implementation architecture
- Appended a section going over the design patterns used and considered (section 2 (Appended) Key Design Choices)
- Appended 6.1 to better reflect the systems that needed to be implemented
- Appended 6.3 to better reflect UC1-UC7 implementation
- Appended sections 10 through 12 covering the distinguishable RBAC feature, frontend implementation, and Docker deployment
- Updated section 6.1 product backlog to reflect completed D3 features including admin dashboard, RBAC, Docker, and frontend
- Updated section 6.4 Deliverable 3 sprint backlog to mark all tasks as completed
- Appended section 7 meeting logs with Meeting 5 and Meeting 6 covering D3 planning and final review
- Extended section 9 test plan summary to include D3 admin security testing
- Updated the README to include Docker deployment instructions and frontend run instructions
- Updated the Postman collection to include admin endpoint tests
- Appended AI acknowledgements section to better represent how AI was used in the making of the app and documentation
- Appended section on advantages and disadvantages of AI use in documentation and app creation
- Appended Test Plan Summary to include JMeter test results

17 (Appended) Team Member Parts

Mahmoud Dahlan: Primarily responsible for backend development and frontend development, including REST API implementation, authentication, RBAC, and database integration. Also contributed to Postman testing and debugging backend issues and overall coordination of deliverables

Yazan Haddad: Primarily responsible for frontend development, including React pages, UI styling and responsible for JUnit test creation and JMeter testing. Also assisted with frontend testing, validation, and improving user experience.

Lama Abdelfattah: Primarily responsible for testing and quality assurance, including Postman test cases, and failure analysis. Also contributed to backend debugging and validating API functionality.

Amanullah Asim: Primarily responsible for deployment and backend development, including implementation of the AI ChatBot. Also contributed to backend debugging and validating API functionality.

Tristan Augustus: Primarily responsible for project management and documentation, including diagrams, sprint planning, meeting logs, and report formatting.